

SANTA CLARA UNIVERSITY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Date: June 2, 2026

I HEREBY RECOMMEND THAT THE THESIS PREPARED UNDER MY SUPERVISION BY

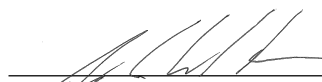
Bryce Kelly [CSEN + ECEN]
Dylan Thornburg [ECEN]

ENTITLED

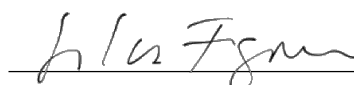
Open Source ARM Cortex-M0 Processor

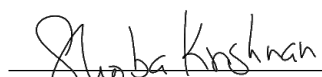
BE ACCEPTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREES OF

BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING
BACHELOR OF SCIENCE IN ELECTRICAL AND COMPUTER ENGINEERING


Thesis Advisor [Dr. Andrew Wolfe [ECEN]]


Thesis Advisor [Dr. Younghyun Cho [CSEN]]


Department Chair [Dr. Silvia Figueira [CSEN]]


Department Chair [Dr. Shoba Krishnan [ECEN]]

Open Source ARM Cortex-M0 Processor

by

Bryce Kelly [CSEN + ECEN]
Dylan Thornburg [ECEN]

Submitted in partial fulfillment of the requirements
for the degrees of
Bachelor of Science in Computer Science and Engineering
Bachelor of Science in Electrical and Computer Engineering
School of Engineering
Santa Clara University

Santa Clara, California
June 2, 2026

Open Source ARM Cortex-M0 Processor

Bryce Kelly [CSEN + ECEN]
Dylan Thornburg [ECEN]

Department of Computer Science and Engineering
Department of Electrical and Computer Engineering
Santa Clara University
June 2, 2026

ABSTRACT

Embedded systems are a key component of modern technology. They are found in areas such as medical devices, IoT, gaming, automobiles, and basic microcontrollers. In this field, low-power, small, and reliable processors are key. The Cortex-M0 is one of these crucial processors. However, compared to other architectures, such as RISC-V or MIPS, the Cortex-M0 is closed-source. This restricts educators, researchers, and smaller start-ups from viewing the implementation or making microarchitecture changes to this design for research and testing. Our project looks to implement an open-source microprocessor, allowing new access to the internals of a behaviorally accurate Cortex-M0 through a Verilog implementation for simulation and synthesis on an FPGA.

Our project resulted in a Verilog implementation of the Cortex-M0 capable of running the ARMv6-M instruction set with a core, AHB-Lite bus matrix, and NVIC. Our implementation was tested to match a commercial Cortex-M0 in both function and timing through comparisons between the Verilog simulation and real Cortex-M0 results from an STMicro STM32F0 Discovery Board. A standard benchmark test, CoreMark, was used to verify and validate the implementation when synthesized on an FPGA. The implementation is open-source licensed through GPLv3 with a public GitHub repository¹ making the code accessible to all for viewing and iterating. This project acts as a minimal implementation of the Cortex-M0, with many optional components not included, along with the Cortex-M0 being the lowest-level Cortex-M Series processor. Our work opens up multiple opportunities through a new computer hardware and research ecosystem open and accessible to all. Potential future projects could continue developing more of the Cortex-M Series, its features, or look into potential improvements that can be made to this processor.

¹GitHub Repository Link: <https://github.com/brkelly6585/Open-Source-ARM-Cortex-M0>

Table of Contents

1 Introduction	1
1.1 Problem Statement	1
1.2 Background or Related Work	1
1.3 Objectives	2
1.4 Our approach	3
2 User Research	4
2.1 Methods	4
2.2 Stakeholder needs	4
2.3 User stories	5
3 Design and Rationale	6
3.1 Design	6
3.2 Use Case Overview	9
3.3 Use Case Descriptions	9
3.4 Use Case Diagram(s)	12
3.5 Class Diagram	13
3.6 Sequence Diagram	14
3.7 Activity Diagram	15
3.8 Component Diagram	16
3.9 Functional requirements	17
3.10 Non-functional requirements	17
3.11 Rationale	18
4 Technologies	19
4.1 System Components	19
5 System Evaluation	20
5.1 Internal Testing	20
5.2 External Testing	21
5.3 Unit Testing	21
5.4 Integration Test(s)	29
6 Implementation Plan	31
6.1 Timeline	31
6.2 Project Risks	33

7 Constraints and Standards	35
7.1 Constraints	35
7.1.1 Time	35
7.1.2 Reliability	35
7.1.3 Legal	35
7.1.4 Available Documentation	36
7.2 Standards	37
7.2.1 Verilog	37
7.2.2 Synthesizable Verilog	37
7.2.3 Cortex-M0 Technical Reference Manual	37
7.2.4 ARMv6-M	37
7.2.5 AMBA 3 APB	38
8 Societal Issues	39
8.1 Ethical	39
8.2 Health and Safety	40
8.3 Manufacturability	40
8.4 Sustainability	40
8.5 Environmental Impact	41
8.6 Usability	41
9 Conclusion	42
10 Glossary of Abbreviations	43
11 Appendices	53
11.1 Source Code	53
11.1.1 M0 Top Module	53
11.1.2 IFID Buffer - Example Buffer	57
11.1.3 Special Data Instruction Decoding - Example Decode	58
11.1.4 AHB Subordinate/Secondary Module	59
11.2 Top Level RTL Synthesis	61

List of Figures

3.1 Top Level Design [17]	6
3.2 Processor Design [17]	7
3.3 Core Pipeline	8
3.4 AHB design	8
3.5 Use Case 1: Installation of Open Source Verilog	9
3.6 Use Case 2: Virtual Simulation of Processor	9
3.7 Use Case 3: FPGA Synthesis of Processor	10
3.8 Use Case 4: Altering Implementation of Processor	10
3.9 Use Case 5: Collaborating on Open Source Design	11
3.10 UML Use Case Diagram	12
3.11 Class Diagram	13
3.12 Sequence Diagram	14
3.13 Activity Diagram	15
3.14 Component Diagram	16
5.1 AHB Main Interface Expected Waveform	22
5.2 AHB Main Interface Results	22
5.3 AHB Secondary Interface Results	23
5.4 Register File tb	24
5.5 Register File tb Results	25
5.6 ALU tb	26
5.7 ALU tb Results	27
5.8 Data Memory tb	28
5.9 Data Memory tb Results	29
5.10 Walking 1's test instructions	30
5.11 Walking 1's results	30
6.1 Fall 2025 quarter Gantt Chart of Timeline	31
11.1 Top Level RTL synthesis from Vivado	61

Chapter 1

Introduction

Acronyms and technical terms are defined in the Glossary (Chapter [10](#)).

1.1 Problem Statement

Embedded systems are a key component in many modern-day devices, ranging from medical devices to automobiles, and within this comes the need for low-power, small, and reliable processors [\[24, 7\]](#). The ARM Cortex-M0 is one of these processors and has become a standard for low-power, cost-effective, and small-area designs because it is ARM's smallest and most baseline processor [\[5\]](#). It is the basis for the ARM Cortex-M Series of processors, which has only grown alongside the development of embedded systems [\[8\]](#). This has led to ARM being a significant designer within the low-power processor design space. However, the closed-source nature of the ARM architecture restricts its flexibility and transparency for developers. This lack of visibility means that these key designs within the industry are not visible for research, educational, or experimental purposes. This poses a great challenge for academic settings which could use the cost-effective nature and versatility of the processor for a wide variety of research or startups looking to take full advantage of the processor's design and potentially build atop it. We propose an open-source implementation of the ARM Cortex-M0 which implements the behavior of the Cortex-M0, and is compatible with the ARM ecosystem. We achieve this through an open-source Verilog codebase used to implement the minimal processor and its related functions along with a complete set of documentation detailing the implementation of the processor and its verification.

1.2 Background or Related Work

Billions of modern devices use embedded systems across multiple industries, including medicine, transportation, industrial equipment, and even automobiles. The main benefits of these systems are their reliability, low power, and

minimal size which allow for fast, powerful systems focused on specific tasks [24]. The ARM Cortex-M architecture family of processors aims to target these specific systems, working within the popular ARM ecosystem and focusing on low-cost performance. Within this family, the ARM Cortex-M0 is the smallest processor, and with the least power consumption along with low cost, it excels in any low-cost environment where embedded systems are used [9].

The Cortex-M0 is part of a broader group of RISC processors. RISC architectures prioritize simple, highly uniform instructions that execute in a single clock cycle. Other examples of RISC-style architectures include MIPS, RISC-V, and OpenRISC [19]. MIPS has been historically important since its early inception in 1985 for computer architecture education due to its simple format and scalable foundation. RISC-V and OpenRISC are more relevant today, with RISC-V especially becoming a major open-standard ISA used in academia and commercial development. Despite these recent advances, ARM still remains the largest embedded computing player worldwide with the Cortex-M processor line retaining 68.25% of the microcontroller market [4].

However, the ARMv6-M ISA is a closed source, so the ARM Cortex-M0 is a closed-source processor. This means its internal design and process are inaccessible to researchers or developers. This absence of transparency causes the Cortex-M0 to lack flexibility, limiting its ability in certain startup businesses, academic research, or hobbyist fields. This distinguishes our project from the current Cortex-M0 product by combining it with the visibility and flexibility seen in other open-source RISC processors while maintaining compatibility with ARM systems. This also allows developers to view the current implementation of a popular architecture and collaborate to make changes or improvements on top of the base processor.

1.3 Objectives

The primary objective of this project is the creation of a complete and tested processor compatible with the ARMv6-M ISA with timing close to the ARM Cortex-M0 processor. In this way, we aim to create an open-source interpretation of a minimal Cortex-M0 processor. To accomplish this goal, we set multiple objectives we planned to meet. First, we needed the Verilog codebase, which is a complete working processor along with its additional modules such as the AHB-Lite bus matrix and NVIC. Along with that, we also aimed to produce both a synthesized implementation that is able to be run on an FPGA development board, and a simulated environment to test performance in a range of different scenarios. With the different implementations, we provided benchmark reports detailing how our design performs compared to other current processors. Finally, we needed to produce complete documentation of our design including specifications, a user manual, and guides to use the processor. We needed to create our own validation tests to show how our processor meets expected functionality and timing on both simulated and synthesized versions, documenting these tests and ensuring they prove our processor meets these desired objectives. All of these come alongside an

open-source release on GitHub under license to ensure accessibility and collaboration among developers.

1.4 Our approach

Our approach was to target smaller components of the processor to make individual development and testing easier. From there we would connect the individual components back into the larger system and test the functions of all components along with their interconnects to ensure the larger system worked before moving to the next component. In this, we could constantly test and document our system and its implementation at each stage. This is important since our implementation requires its own verification and validation tests to showcase its functionality and prove it reliably matches expected behavior. These smaller tests worked to show current functionality while being able to be reused to test future iterations. This results in a system that is better than existing solutions, since it gives access to the tests used to validate the behavior. As such, we focused more on consistent testing and validation, with accessibility and readability being other keys to our development to support the validation and implementation as a whole. Overall, we wanted to approach the project cautiously, making sure that our system was tested along the way so we could better document and explain how or why our system meets the standard for ARMv6-M ISA while detailing our unique implementation [15].

Chapter 2

User Research

2.1 Methods

To ensure that our design would meet the needs of our users, we employed a mix of professional conversations and reviews of other research material. This allowed us to see what currently existed along with what is typically used by professors or industry developers when considering embedded systems or processor-level research. First, we analyzed what was currently available through current tools and software, following ARM documentation along with synthesis releases [16]. Along this path, we found all ARM-provided designs were preconfigured and undisclosed. This also means we did not use any designs or components from these resources in our implementation. This highlights a real gap in current technology where an openly viewable or editable design is not available. Along with that, our discussions with various professors supported the demand for an open-source implementation. Access to the design opens the door to more teaching and research opportunities as direct changes can be made to gate or module-level components. This allows for more tests to be done on internal components, opening the door to testing internal changes which are not possible without open access to the design. Along with that, the ability to view internals during research is valuable since it becomes possible to observe internal changes during normal operation and look for possible improvements.

2.2 Stakeholder needs

One of our key stakeholders is professors or students in academia. Professors or students who are pursuing research or looking to learn more within the field of computer architecture will often use preexisting systems to test or compare against others or as a starting point for modifications. In this, they will need high flexibility and visibility into whichever system they are using to fit with their research goals, and to properly monitor responses of tests so they can observe what they are researching. Our system prioritizes their access into the system internals through being

completely open-source, which allows the researcher to alter or inspect any part of the system, and allows for multiple tests which could not be run on a closed-source processor. Another aspect is easy configuration and low-cost methods for testing designs. Our system prioritizes this through easy FPGA synthesis and again, open access to internals, where these designers can configure the processor however they want by changing the design's function or timing to meet their needs. Meanwhile, FPGA synthesis supports low-cost usage since FPGAs are reconfigurable, allowing for multiple tests without the need to constantly spend on new hardware devices.

2.3 User stories

- As a student, I want to work on developing or researching the ARM Cortex-M0, so I can get experience working on the architecture of a low-level processor.
- As a researcher, I want to create a low-cost system, so I can conduct research without overspending.
- As a researcher, I want to see the internal operation of a low-cost processor, so I can better understand what it does and find potential improvements.
- As a researcher, I want to be able to make changes to parts of the processor I am studying, so that I can simulate them and see their effects on the processor as a whole.
- As a researcher looking at development, I want clear documentation of the processor I am using so I understand how it works and better understand where potential improvements can be made.
- As a researcher looking at development, I want to be able to see and download the Verilog of the processor I am using, so that I can better understand its functions or make changes.

Chapter 3

Design and Rationale

3.1 Design

The figure below comes from ARM Cortex-M0 Processor datasheet and showcases the use of an M0 processor inside a larger embedded system design [17]. As such, our design fits similarly into embedded systems. Utilizing the AMBA 3 AHB-Lite interface, our system is designed to interface with both memory and external peripherals through a standard protocol. This means our system can work within similar systems as the Cortex-M0 or M0+ due to the similar interface protocols and interrupt connections.

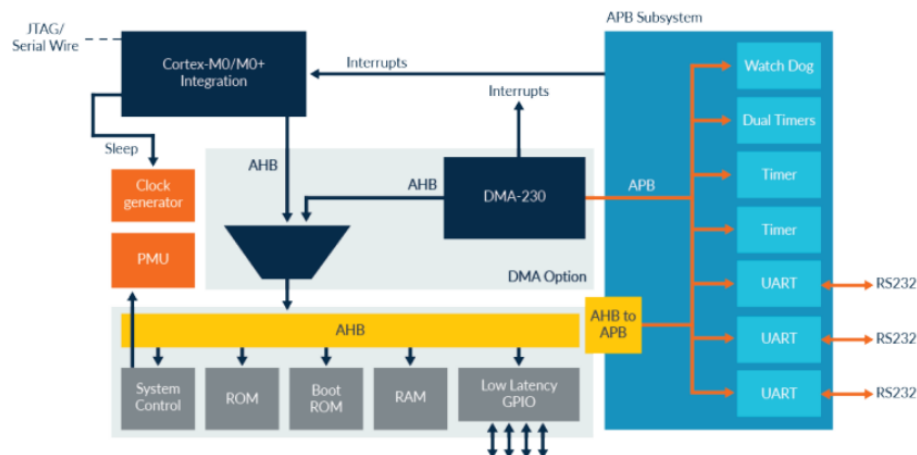


Figure 3.1: Top Level Design [17]

Moving into the processor itself, the following figure showcases a full Cortex-M0 core along with optional components as detailed in the Technical Reference Manual [14]. To build an initial but expansive ecosystem, our design focuses primarily on the main CPU core along side the NIVC and AHB-Lite bus matrix interface. This extensible design fits our use cases as it exposes the core components of the processor which are necessary for basic usage while

leaving the other components for further research. Part of the justification for this design also came down to time constraints as we wanted to ensure the components we did create were properly tested and validated.

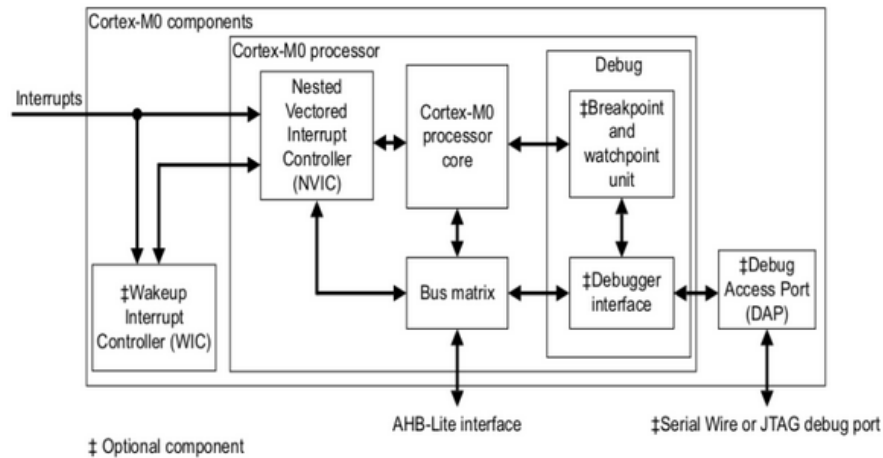


Figure 3.2: Processor Design [17]

Looking into the Cortex-M0 processor core, the following Figure 3.3 shows the main structure detailed in the Cortex-M0 Datasheet [17]. This details a 3-stage pipeline with fetch, decode, and execute phases. To start, the Fetch phase reads in a 32-bit word from memory, which means the core typically fetches 2 instructions at a time since most ARMv6-M instructions are 16-bit halfwords. This means the processor will prefetch its next instruction. After fetching, the instructions go through a 1-cycle buffer into the Decode phase where control signals are generated based on instruction encodings listed in the ARMv6-M manual [15]. These control signals are used to read register values and generate addresses for memory functions before moving through another buffer into the Execute phase where the ALU, Data Memory, and Branching units will perform the operations detailed by the control signals before writing any data back. Due to the pipelined design, many instructions only take 1 cycle in the pipeline to complete. However, memory and branch instructions have additional delays since the AHB-Lite Interface incurs a 1-cycle penalty and branching requires draining the entire pipeline.

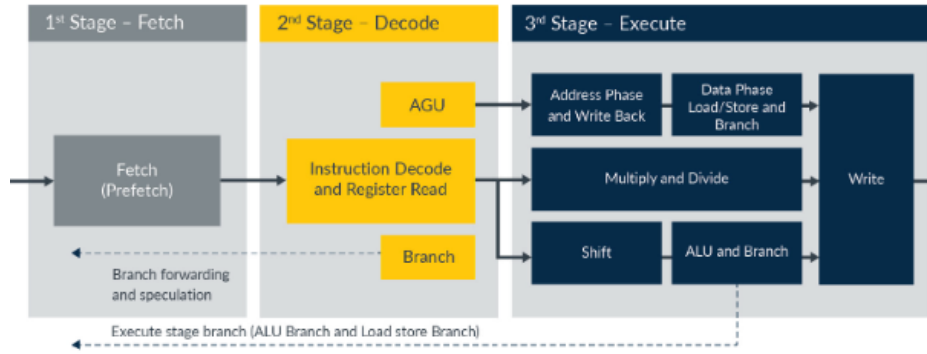


Figure 3.3: Core Pipeline

Moving on to the next key component, the AHB-Lite interface operates as a 2-stage pipeline between the Manager and Subordinate modules [13]. The Manager will be stored within the internal Bus Matrix of the processor and interface with external Subordinates, including memory. The interface uses a decoder and mux selector to decide which Subordinate data will be returned to the Manager with set address regions defined in the Cortex-M0 Technical Reference Manual being used to decide which Subordinates are selected [14].

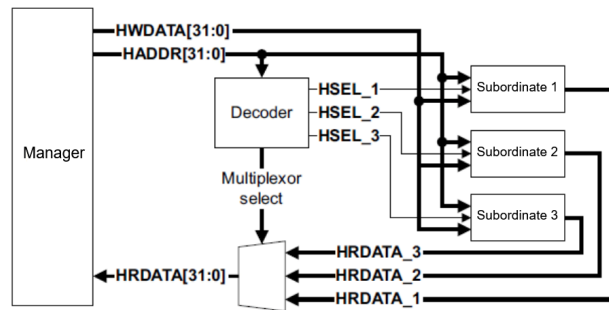


Figure 3.4: AHB design

The final component for a basic Cortex-M0 is the NVIC. The ARM documentation does not detail much about the Cortex-M0 NVIC. Thus, we instead followed a more general description and implementation for the Cortex family NVIC [6, 25]. The key aspect for our implementation is that it uses a 32-bit IRQ word to interface with all 32 external interrupt requests and has 4 priority levels as detailed in the Cortex-M0 Reference Manual [14].

3.2 Use Case Overview

The main way the upcoming use cases were derived from the user stories was through how a user would be able to access our system. Since many of the user stories focused on different ways to run or access our system, these use cases go through the steps of using our system for the purpose of research, or if they wanted to develop something new using our implementation of the ARM Cortex-M0 processor. The other user story is related to someone collaborating on the processor and looking to improve upon the implementation. It focuses on the steps to both test and upload the change(s) to the open-source implementation.

3.3 Use Case Descriptions

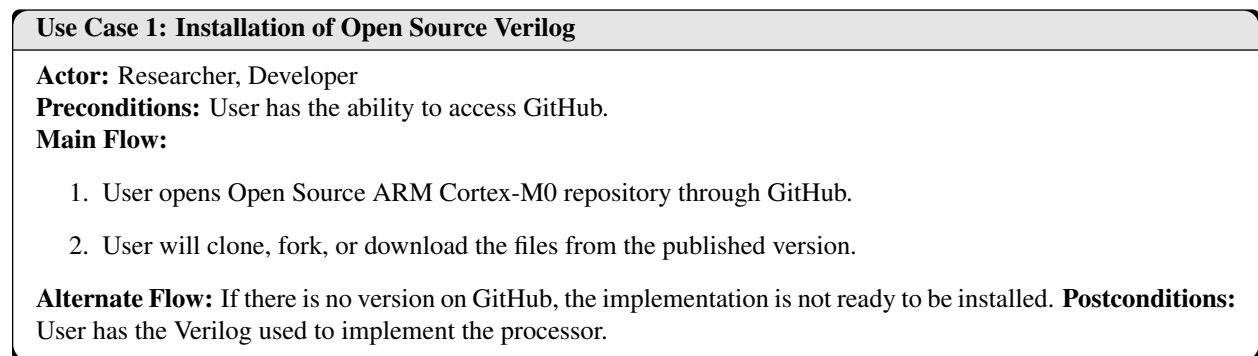


Figure 3.5: Use Case 1: Installation of Open Source Verilog

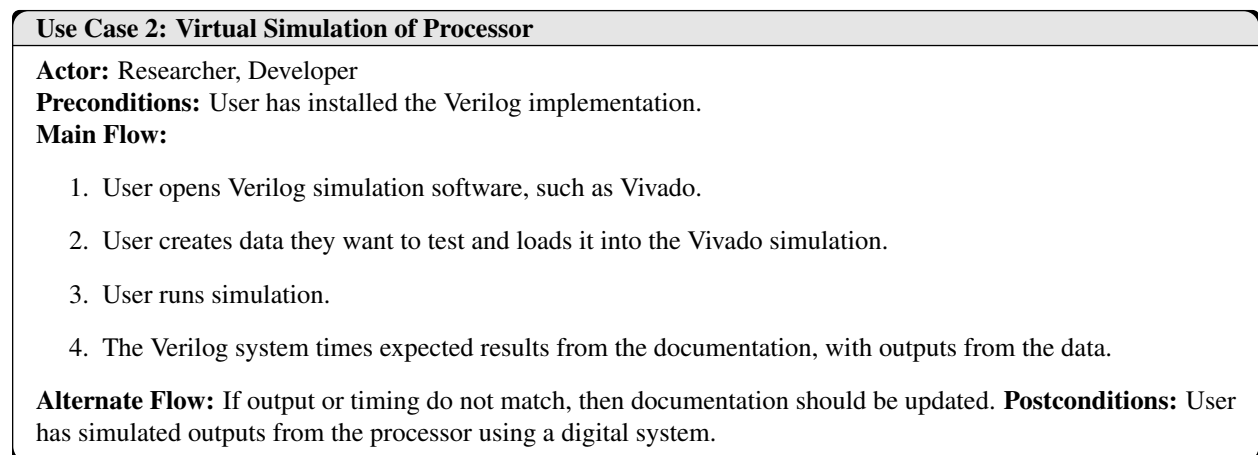


Figure 3.6: Use Case 2: Virtual Simulation of Processor

Use Case 3: FPGA Synthesis of Processor

Actor: Researcher, Developer

Preconditions: User has installed the Verilog implementation, and has FPGA capable of loading Verilog synthesis

Main Flow:

1. User opens Verilog synthesis software, such as Vivado.
2. User synthesizes Verilog and sets the implementation for the target FPGA.
3. User loads the synthesized Verilog bitstream onto the FPGA.
4. User loads instructions and data into the FPGA to physically operate the design.
5. System outputs results following the expected timing.

Alternate Flow: If timing does not match, then documentation should be updated. **Postconditions:** User has loaded the Verilog implementation onto an FPGA to run on hardware.

Figure 3.7: Use Case 3: FPGA Synthesis of Processor

Use Case 4: Altering Implementation of Processor

Actor: Researcher, Developer

Preconditions: User has installed the Verilog implementation.

Main Flow:

1. User creates their own Verilog module to implement a function of ARM Cortex-M0.
2. User opens a simulation software, such as Vivado.
3. User replaces module from open-source implementation with their implementation.
4. User runs simulation on the new implementation.
5. User compares the result to current and expected results.

Alternate Flow: If the Verilog can no longer be simulated, the user should run with the open-source implementation. **Postconditions:** User has added personal changes to the base open-source implementation and simulated them.

Figure 3.8: Use Case 4: Altering Implementation of Processor

Use Case 5: Collaborating on Open Source Design

Actor: SCU Student

Preconditions: User has installed the Verilog implementation.

Main Flow:

1. User researches current ARM Cortex-M0 processor.
2. User implements or refines an additional feature of the ARM Cortex-M0 from open-source implementation.
3. User simulates Verilog in simulation software such as Vivado.
4. User synthesizes Verilog on FPGA.
5. User documents results.
6. User submits changes and results to GitHub for Open Source Implementation.

Alternate Flow: If addition does not have proper documentation or does not match ARM Cortex-M0 specifications, it is not added to the implementation. **Postconditions:** User has updated the Open Source Implementation.

Figure 3.9: Use Case 5: Collaborating on Open Source Design

3.4 Use Case Diagram(s)

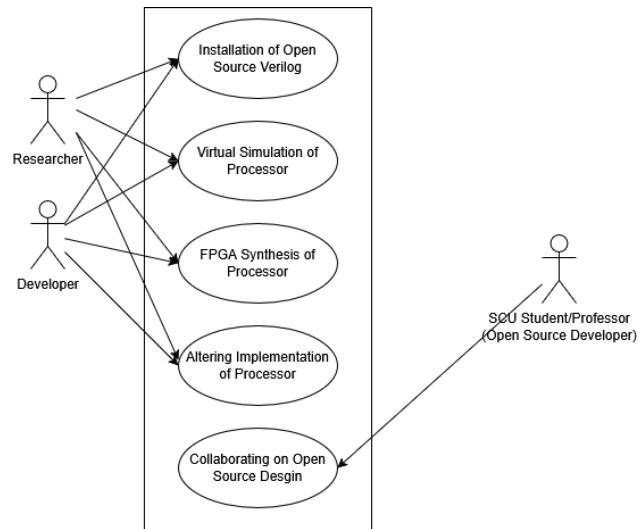


Figure 3.10: UML Use Case Diagram

3.5 Class Diagram

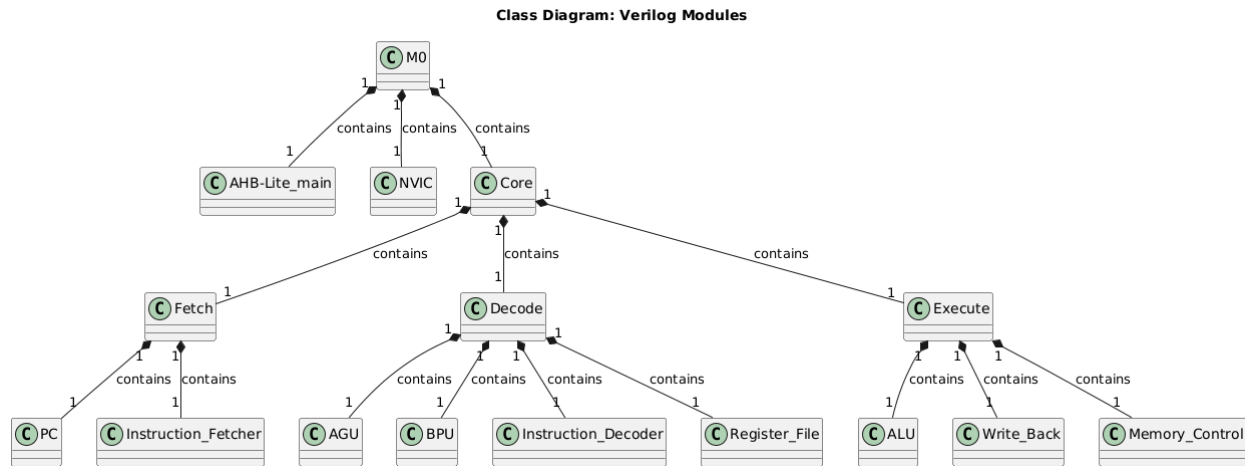


Figure 3.11: Class Diagram

Since our project is in Verilog, there are not traditional classes. Instead our project is structured through modules. In the case of our project, there is only one type of module which is connected or contained within another module. For example, the Instruction Decoder is contained within the Decode stage, and a buffer is within the core. Starting from the very top, we have the M0 itself, this is the top-level module that will be the processor itself. Below that are the next-level architecture modules, the AHB interface, NVIC, and core. Both the AHB interface and NVIC have limited submodules when it comes to the M0 itself, since their job is to connect with modules outside of the M0. Following the core, it breaks down into three key stages of a pipeline: Fetch, Decode, and Execute. Within each stage are the modules necessary to process the instructions and make sure their data is saved properly in either registers or memory.

3.6 Sequence Diagram

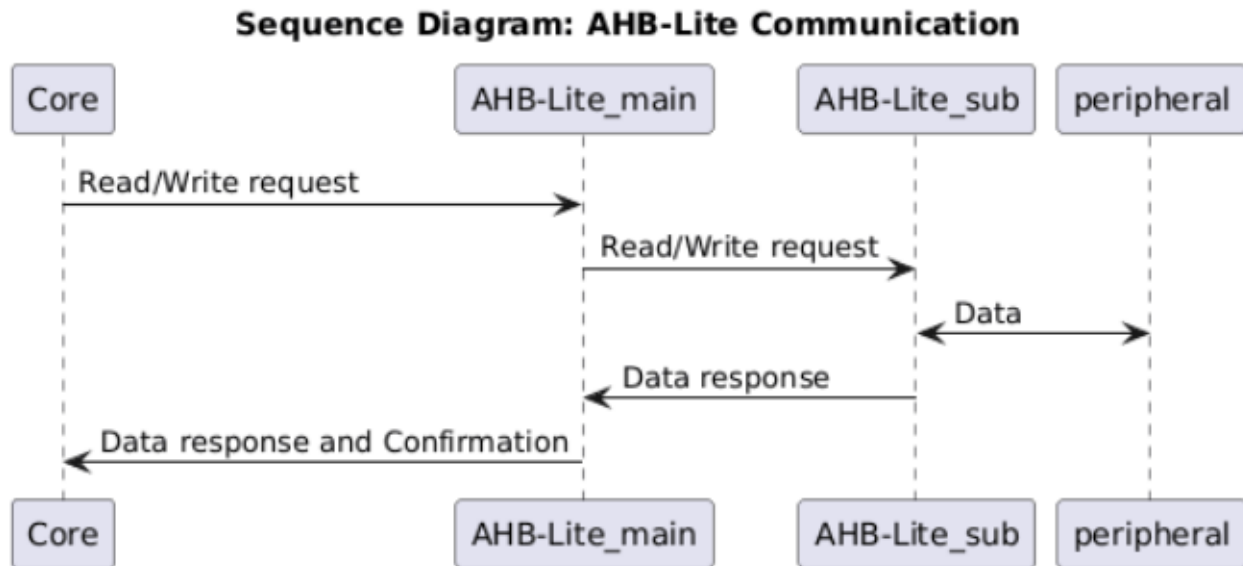


Figure 3.12: Sequence Diagram

Above is the sequence diagram for AHB-Lite interfacing. First the core will send out a request to the main AHB controller. This request can be either a read or write, along with the address to read/write in memory. Write requests send data along with them to change the memory. From there the request is sent from the main AHB controller to a subcontroller attached to the memory module, or other peripheral. This submodule then interfaces with the peripheral to get the requested data and send it back along with a confirmation to the core.

3.7 Activity Diagram

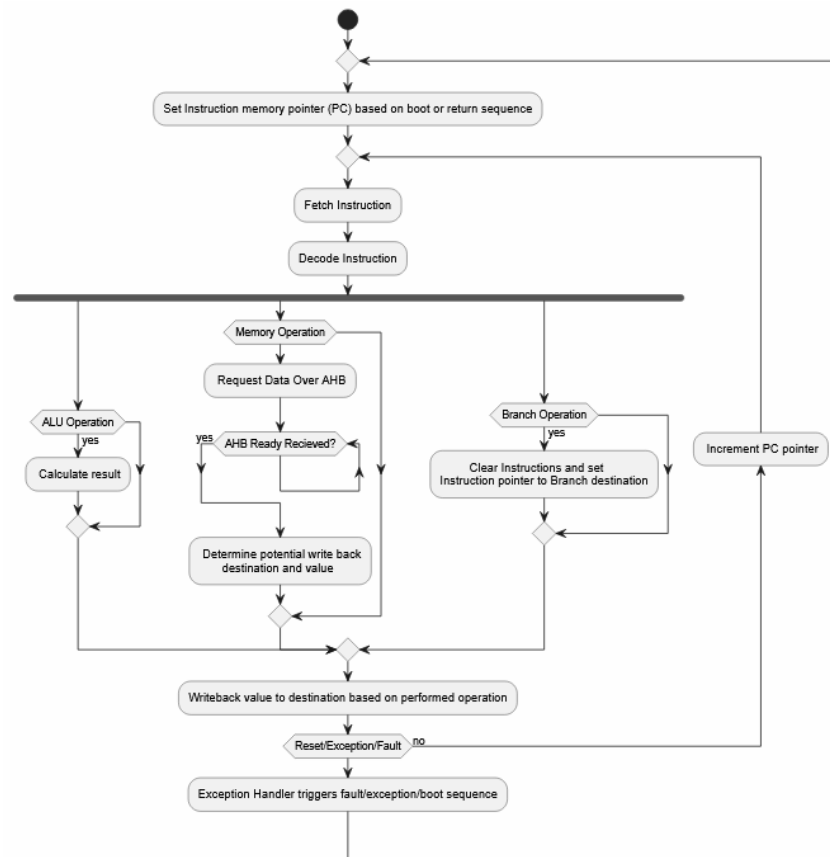


Figure 3.13: Activity Diagram

This activity diagram represents the flow of activity for instructions inside of the core, which is the main processing sector for instructions and their results. Going through, the core first starts by reading the very first instruction in memory and entering a loop that continues to read instructions until the processor is either forced to stop, or until the end of instruction memory is reached. Within the loop, the core starts by fetching a set of instructions before moving to decode them. There, it will break down the instruction and move the required data into the execution phase. Once in execution, the instruction will perform its task and write any changes back to registers, or send a request out on the AHB interface as discussed in section 3.6. At the same time, if data is seen coming in on the AHB interface, then it will be bypassed through and written to the registers. This will then continue to repeat until reset.

3.8 Component Diagram

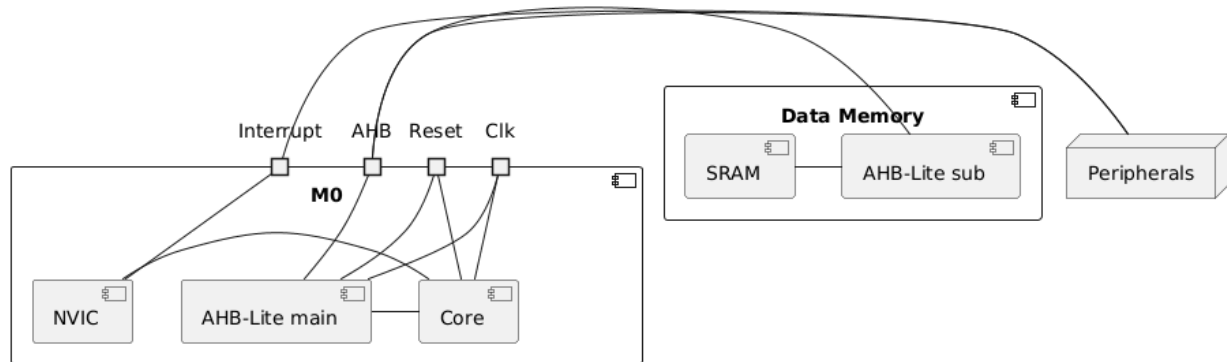


Figure 3.14: Component Diagram

The above component diagram mostly works to show the connections between the M0 Processor we are designing and the memory block or peripherals that will be used to interact with the processor. The first connection is the AHB interface and the interrupt lines, which are used to connect with Peripherals, particularly the Data Memory block which will be used to hold data and instructions within SRAM.

3.9 Functional requirements

Our system looks to replicate the functions of the ARM Cortex-M0 processor, and thus, our system must be able to read and run all ARM Cortex-M0 instructions with these instructions coming through a memory bus. This functional requirement entails proper decoding of the ARMv6-M instruction set, which details the structure, or encoding, and function of each instruction that is capable of being run on the ARM Cortex-M0. On top of that, our system must be able to read and write data through an AHB memory interface. This again comes from the ARM Cortex-M0 functions, which require reading from memory in order to fetch instructions, and require both reads from and writes to memory for certain instructions in its instruction set. With the memory bus, our system must be able to process interrupts just like the current ARM Cortex-M0 processor. This requires the implementation of an NVIC unit inside our processor and an Interrupt Handler to be able to organize and run incoming interrupts. To be a true implementation of the ARM Cortex-M0, the timing of our system must match the current processor. This means that both cycle counts of instructions and cycle time must exactly match the current processor and must be tested to ensure our implementation runs at the same rate on real-world hardware.

Alongside the processor-specific functional requirements, our system must have full documentation, including timing, details about use, and specifications about how it runs. This is fundamental to our system since the objective is to be open-source, meaning it requires proper documentation to show functionality and make it possible for others to use. Finally, our system must follow open-source licensing; this is crucial to our project being an open-source implementation as without it our system would not have the legal licensing to be used by others as they wish.

3.10 Non-functional requirements

First, our system should be easy to use, meaning it is easy to set up and run on different systems. Our system should just require downloading the implementation and then should be able to be either simulated virtually or run on an FPGA, with the prerequisite that the FPGA can actually run synthesized Verilog. In addition to this, our documentation shall have high readability. As such, the documentation will be easy to navigate between sections and will have connected information sectioned together. The documentation will also include only relevant information about the functions and specifications of our system. Finally, our system shall be accessible. That is, the Verilog code of the implementation will be easy for others to view and download for their own use without excessive barriers. The documentation should be equally accessible as the processor itself, meaning someone using the M0 implementation can access the documentation from the same place.

3.11 Rationale

The system overall was designed this way due to the popularity and widespread use of the Cortex-M0 processor. Low-level processors in general are crucial to embedded systems and the M0 is one of the most popular designs. Due to this, it serves as a good base for an open-source processor interpretation. On top of that, as a low-level processor, it leaves room for more components to be built on top of it again, serving as a baseline open-source design for other contributors to expand upon. Finally, our rationale behind both simulation and synthesis designs came from ensuring our design could be a physical device since simulation alone could be used for research but it does not ensure that the processor design is feasible in the real-world.

Chapter 4

Technologies

4.1 System Components

The core system components that we used were AMD Vivado, Verilog, and the Digilent Nexys A7-100T FPGA, Field-Programmable Gate Array, board [12, 20, 10]. Verilog was the language we developed our project in since this is an RTL, Register-Transfer Level, implementation. Vivado is the IDE, simulation, and synthesis tool used to implement and test our design. Finally, the Nexys A7-100T FPGA trainer board was the FPGA we synthesized and tested our design on.

For testing, we also used an STM32F0 Discovery Board with a mounted Cortex-M0 and the Keil IDE [11, 3]. The STM board was primarily used to test the on-board Cortex-M0 for comparison against our implementation. We would use Keil to connect with and run ARM Assembly instructions on the Cortex-M0 through the STM board and then view the register results.

Chapter 5

System Evaluation

In general, many of our internal tests focus on functional requirements. Our testing involves all instruction executions, pipeline behavior, memory/peripheral interface, interrupt handling, and FPGA validation. These tests are meant to test the individual purpose of the Core, NVIC, and AHB-Lite interface modules of the processor. Instruction executions and FPGA validation, however, are more general tests, meant to ensure that the processor as a whole with all modules combined is functioning as intended. Within all of these categories we will focus on functional tests alongside timing tests since both are critical to determining the capabilities of the processor and ensuring it meets design requirements.

Since many of our nonfunctional requirements are based on subjective views, such as readability or ease of use, these will be tested externally. This gave us a broader view to ensure we are able to properly meet these requirements without internal bias.

5.1 Internal Testing

All Verilog simulation testing was performed on AMD Vivado. We would compare our results for instruction executions to a real M0 on an STMicro Discovery board (STM32F0) to ensure they are performing as expected. This was generally done by generating a set of ARMv6-M instructions to test a mix of functions in various situations. Then we would run that set through the Keil IDE, a tool for debugging Cortex-M series microcontrollers, with the STM32F0. This would let us view the register values after our instruction set was run, which we could then compare against our simulation results in Vivado.

To analyze cycle counts, we would first refer to manuals to see if waveforms match. This is particularly true for the AHB-Lite interface where we can get diagrams for the protocol and match them in simulation. Data functions could also be tested this way since their cycles are detailed in the Technical Reference Manual. This method of analyzing timing is helpful for making sure data updates on the correct clock cycle.

However, we also needed to run real-world comparisons using an FPGA against the real M0 processor to ensure our

implementation was synthesizable while maintaining accuracy. Our way of testing this actual runtime of cycles was to time how long a swath of instructions takes on the real Cortex-M0. This can be done by looping a few lines of code 100+ or 1000+ times, adjusting for timescale differences between real life and the simulation, and then comparing the timing results of those same instructions in simulation vs the measured real-time. Combining the data from our real timing and the cycle count tests allowed us to properly ensure that the processor meets timing requirements. These tests are necessary, primarily for testing pipeline behavior, memory interface, and interrupt control.

When it comes to instruction executions, our testing focused more on meeting the functional requirements of each instruction, which means testing each function in different scenarios and the different hazards they may cause to ensure they run as expected. We test this by simulating the instructions through our processor and then comparing values stored in both registers and memory to values we get from running the same instruction set on the STM32F0 board. We can create these instruction tests by compiling C code into ARM assembly, making it easier to test more complex scenarios.

Finally, for FPGA verification, we used the open-source CoreMark CPU benchmark software [1]. EEMBC's CoreMark is a standard test for embedded-systems-level processors, going through certain common operations such as matrix math, linked lists, and state machines. The tests themselves are created through compiled C code, testing our implementation's ability to load code as expected of a typical ARMv6-M compiler and the setup required to run a typical program [2]. Through this benchmark, we could confirm both timing and function of our FPGA implementation, verifying its accuracy through multiple iterations of the CoreMark tests, utilizing its built-in error checking.

5.2 External Testing

For external system testing we gave our system out to current projects or boards that already implement the Cortex M0. By replacing the existing board with our implementation, the systems should still function the same. This was our primary source of external testing of functional testing. However, for non-functional requirements, we did more testing through interviews or questionnaires about our documentation. These questions focused on ensuring our documentation would meet the quality and readability that we expected for our project.

5.3 Unit Testing

Since our project primarily uses Verilog, our main method of unit testing was with Verilog testbenches. We would design and use specific testbenches to test the key individual modules of our system. This included tests for the Register File, Instruction Decode, Data Memory, ALU, and Instruction Memory blocks within the core. We also had tests for the individual Main (Manager) and Secondary (Subordinate) AHB Interfaces within the Main AHB-Lite Memory

structure. Finally, we tested the Interrupt Controller and Priority Arbiter separately within the NVIC to guarantee each individual module was working as expected.

Below are our unit tests for both the Main and Secondary AHB-Lite Interfaces. These must be separate since we need to test that the Main can interact with multiple secondary interfaces. On top of that, they have different synchronization with the clock. In this, we would see that both the Main and Secondary match their expected timing diagrams and are able to send data through their modules. Since the interfaces rely heavily on timing, we show the waveforms as the result and against the expected diagrams. In the secondary results, the primary line we are looking at is rdata since it is the line most used by the Secondary units to transmit data between itself and the Main.

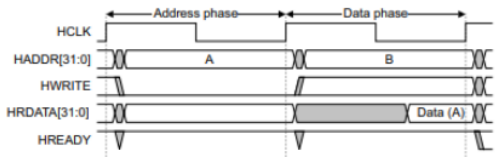


Figure 3-1 Read transfer

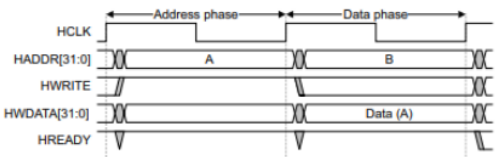


Figure 3-2 Write transfer

Figure 5.1: AHB Main Interface Expected Waveform

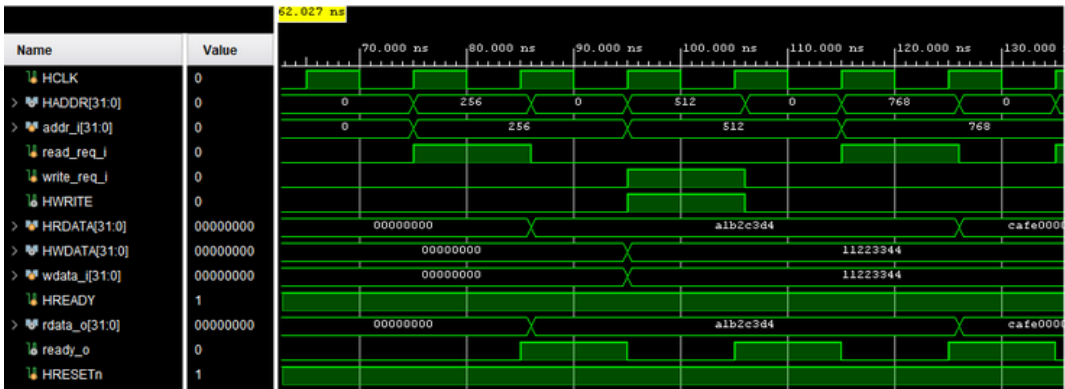


Figure 5.2: AHB Main Interface Results

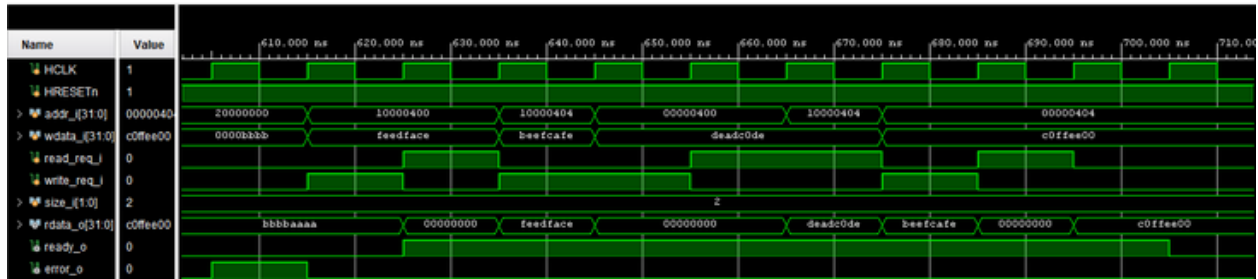


Figure 5.3: AHB Secondary Interface Results

Below are the tests of the Register File, Data Memory, and ALU. These are important since they are key modules within the Core that handle data and execution, and must align with the expected cycles. As such, these tests make sure that the values are properly updated, along with ensuring that they get their updates on the expected clock edge.

In the following Register File tests, we go through the Main function of simply writing along with 2 edge cases testing forwarding and writing to multiple different registers. This tests both the typical use case and extremes of the module.

```

// -----
// Test 1: Write to R3, then read it
// -----
$display("\n-- Test 1: Write 0xAAAA_BBBB to R3 --");
wd_WB = 32'hAAAA_BBBB;
Rd_WB = 4'd3;
we_WB = 1;

@(negedge clk); // write happens here
#1
we_WB = 0;

// Read back
Rn = 3;
#1;
$display("Rn_o = %h (expected AAAA_BBBB)", Rn_o);
// -----
// Test 2: Forwarding check
// -----
$display("\n-- Test 2: Forwarding behavior --");
wd_WB = 32'h1234_5678;
Rd_WB = 4'd5;
we_WB = 1;
// Set read ports to match Rd_WB
Rn = 5;
Rm = 5;
Rt = 5;
// Before negedge: forwarding should give wd_WB
#1;
$display("Forwarded Rn_o = %h (expected 1234_5678)", Rn_o);

@(negedge clk); // commit write
#1
we_WB = 0;
// After write: should still read same value from register
#1;
$display("Post-write Rn_o = %h (expected 1234_5678)", Rn_o);

// -----
// Test 3: Multiple writes
// -----
$display("\n-- Test 3: Write multiple registers --");

for (idx = 0; idx < 4; idx = idx + 1) begin
    wd_WB = 32'h1000_0000 + idx;
    Rd_WB = idx;
    we_WB = 1;
    @(negedge clk);
    we_WB = 0;
end

// Read them back
for (idx = 0; idx < 4; idx = idx + 1) begin
    Rn = idx;
    #1;
    $display("Reg[%0d] = %h (expected %h)", idx, Rn_o, 32'h1000_0000 + idx);
end

$display("\n=== Testbench Complete ===");
$finish;
end

```

Figure 5.4: Register File tb

```

=== Register File Testbench Start ===

-- Test 1: Write 0xAAAA_BBBB to R3 --
Rn_o = aaaabbbb (expected AAAA_BBBB)

-- Test 2: Forwarding behavior --
Forwarded Rn_o = 12345678 (expected 1234_5678)
Post-write Rn_o = 12345678 (expected 1234_5678)

-- Test 3: Write multiple registers --
Reg[0] = 00000000 (expected 10000000)
Reg[1] = 10000001 (expected 10000001)
Reg[2] = 10000002 (expected 10000002)
Reg[3] = 10000003 (expected 10000003)

=== Testbench Complete ===

```

Figure 5.5: Register File tb Results

In the following ALU tests, we go through both specific commonly used tests and flags, but also an assortment of random ALU inputs. We do this since there are many different types of inputs we can put through the ALU, so by having this variety of random tests, we test combinations we may not have thought of and ensure the module works in a variety of ways.

```

// -----
// ADD (example opcode 0)
// -----
$display("\n-- Test ADD --");
run_op(ADD, 32'h0000_0005, 32'h0000_0003, 1);

// Overflow ADD
run_op(ADD, 32'h7FFF_FFFF, 32'h0000_0001, 1);

// -----
// SUB (example opcode 1)
// -----
$display("\n-- Test SUB --");
run_op(SUB, 32'h0000_0005, 32'h0000_0003, 1);

// Negative result
run_op(SUB, 32'h0000_0003, 32'h0000_0005, 1);

// -----
// AND (example opcode 2)
// -----
$display("\n-- Test AND --");
run_op(AND, 32'hFFF_0000, 32'h0F0F_F0F0, 0);

// -----
// OR (example opcode 3)
// -----
$display("\n-- Test OR --");
run_op(EOR, 32'hAAA_5555, 32'h0F0F_0F0F, 0);

// -----
// Zero flag check
// -----
$display("\n-- Test Zero Flag --");
run_op(ADD, 32'h0000_0000, 32'h0000_0000, 1);

// -----
// Randomized stress test
// -----
$display("\n-- Random Stress Test --");
repeat (10) begin
    run_op($random, $random, $random, 1);
end

$display("\n== Testbench Complete ==");

```

Figure 5.6: ALU tb


```

=== arthALU Testbench Start ===

-- Test ADD --
OP=b A=00000005 B=00000003 | Result=00000008 N=0 Z=0 C=0 V=0
OP=b A=7fffffff B=00000001 | Result=80000000 N=1 Z=0 C=0 V=1

-- Test SUB --
OP=a A=00000005 B=00000003 | Result=00000002 N=0 Z=0 C=0 V=0
OP=a A=00000003 B=00000005 | Result=fffffffe N=1 Z=0 C=1 V=0

-- Test AND --
OP=0 A=ffff0000 B=0f0ff0f0 | Result=0f0f0000 N=1 Z=0 C=1 V=0

-- Test OR --
OP=1 A=aaaa5555 B=0f0f0f0f | Result=a5a55a5a N=1 Z=0 C=1 V=0

-- Test Zero Flag --
OP=b A=00000000 B=00000000 | Result=00000000 N=0 Z=1 C=0 V=0

-- Random Stress Test --
OP=4 A=c0895e81 B=8484d609 | Result=ffe044af N=1 Z=0 C=0 V=0
OP=3 A=06b97b0d B=46df998d | Result=000035cb N=0 Z=0 C=1 V=0
OP=5 A=89375212 B=00f3e301 | Result=8a2b3514 N=1 Z=0 C=0 V=0
OP=d A=3b23f176 B=1e8dcd3d | Result=blea071e N=1 Z=0 C=0 V=0
OP=d A=462df78c B=7cfde9f9 | Result=8e60332c N=1 Z=0 C=0 V=0
OP=6 A=e2f784c5 B=d513d2aa | Result=0de3b21a N=0 Z=0 C=0 V=0
OP=5 A=bbd27277 B=8932d612 | Result=45054889 N=0 Z=0 C=1 V=1
OP=f A=793069f2 B=e77696ce | Result=18896931 N=0 Z=0 C=0 V=0
OP=8 A=e2ca4ec5 B=2e58495c | Result=22484844 N=0 Z=0 C=0 V=0
OP=d A=96ab582d B=b2a72665 | Result=b00b77c1 N=1 Z=0 C=0 V=0

=== Testbench Complete ===

```

Figure 5.7: ALU tb Results

In the following tests for the Data Memory block, we focused more on the control lines. This is because this module is more of a controller for the AHB interface, so our unit tests tested both load and store conditions, ensuring that there are no errors in the output control signals.

```

// -----
// Test 1: Store request should NOT stall
// -----
$display("\n-- Test 1: Store request --");
issue_store(32'h1000_0000, 32'hDEAD_BEEF);

@(posedge clk);
if (!write_req_o) begin error = 1;
    $display("ERROR: write_req_o not asserted!");
end

if (addr_o != 32'h1000_0000) begin error = 1;
    $display("ERROR: addr_o incorrect");
end

// Simulate ready
ready_i = 1;
@(posedge clk);
ready_i = 0;

if(error ==0)
    $display("PASS");

// -----
// Test 2: Load request SHOULD stall until ready
// -----
$display("\n-- Test 2: Load request stall behavior --");
issue_load(32'h2000_0000, 5);

@(posedge clk);
if (!read_req_o) begin error = 1;
    $display("ERROR: read_req_o not asserted!");
end

if (!stall_o) begin error = 1;
    $display("ERROR: load should assert stall!");
end

// Hold stall for 3 cycles
repeat (3) @(posedge clk);

// Now release ready
ready_i = 1;
@(posedge clk);
ready_i = 0;

// After ready, stall must drop and wb_valid must assert
@(posedge clk);
if (stall_o) begin error = 1;
    $display("ERROR: stall_o did not clear after ready!");
end

if (!wb_valid_o) begin error = 1;
    $display("ERROR: wb_valid_o not asserted after load completion!");
end

if (Rd_o != 5) begin error = 1;
    $display("ERROR: Rd_o mismatch!");
end

if(error ==0)
    $display("PASS");

```

Figure 5.8: Data Memory tb

```

=== Data Memory Control Test Start ===

-- Test 1: Store request --
PASS

-- Test 2: Load request stall behavior --
PASS

=== Test Complete ===

```

Figure 5.9: Data Memory tb Results

5.4 Integration Test(s)

Our project involves multiple Verilog modules being combined into larger systems. This led to multiple integration tests for all the combinations of modules required. The main tests we completed were for the Core, AHB-Lite interfaces, Core & AHB-Lite interface, the NVIC, and all components combined together. Within each of these broader tests for the components, there are more specific tests to ensure each function within the components was functioning correctly. Within the Core we would use tests to individually test the integration of Data instructions, Branch instructions, Memory instructions, and System instructions. The AHB-Lite tested the integration of the Main and sub interfaces to ensure the Main interface could communicate with multiple subinterfaces. For the Cortex-M0 system as a whole we used more standard tests and validation to ensure the Core, AHB-Lite interface, and NVIC operated correctly together. These tests included Walking 1's along with Walking Addresses for testing memory along with running various compiled C code to ensure the system fulfilled functional requirements [23]. This testing also included tests run on an FPGA, the main purpose was to ensure the synthesized version of our project continued to work as expected along with testing that all of our modules were able to be synthesized together. For all tests, we used Verilog testbenches meant for simulation combined with tasks for individual tests. This allowed us to automate testing by using multiple tasks within a single testbench file. We would build testbenches for each level of the system so we could test at each step of development along with running all tests at the end to ensure the integrated system worked as expected on all levels.

Below is the Walking 1's and Walking Address tests used to test Data, Branch, and Memory instructions in the Core along with the integration of the Core and AHB-Lite interface in the top level M0 module.

```

/* Walking Is & Walking Address*/
Instructions[0] = 16'b00100_000_00000000; // MOVs r0, #0
Instructions[1] = 16'b00100_001_00000001; // MOVs r1, #1
Instructions[2] = 16'b00100_011_00010000; // MOVs r3, #16
Instructions[3] = 16'b01100_00000_000_001; // STR r1 ← {r0}
Instructions[4] = 16'b01101_00000_000_010; // LDR r2 ← {r0}
Instructions[5] = 16'b010000_1010_010_001; // CMP r1, r2
Instructions[6] = 16'b1101_0001_00000101; // BNE +4
Instructions[7] = 16'b00000_00001_001_001; // LSLS r1, r1, #1
Instructions[8] = 16'b000111_0_100_000_000; // ADD r0, #3
Instructions[9] = 16'b000111_1_001_011_011; // SUB r3, #1
Instructions[10] = 16'b1101_0001_11110111; // BNE -8
Instructions[11] = 16'b11100_1111111110; // B . (branch -1)
Instructions[12] = 16'b11100_1111111110; // B . (branch -1)

```

Figure 5.10: Walking 1's test instructions

>  [0][31:0]	00000001
>  [1][31:0]	00000002
>  [2][31:0]	00000004
>  [3][31:0]	00000008
>  [4][31:0]	00000010
>  [5][31:0]	00000020
>  [6][31:0]	00000040
>  [7][31:0]	00000080
>  [8][31:0]	00000100
>  [9][31:0]	00000200
>  [10][31:0]	00000400

Figure 5.11: Walking 1's results

Chapter 6

Implementation Plan

6.1 Timeline

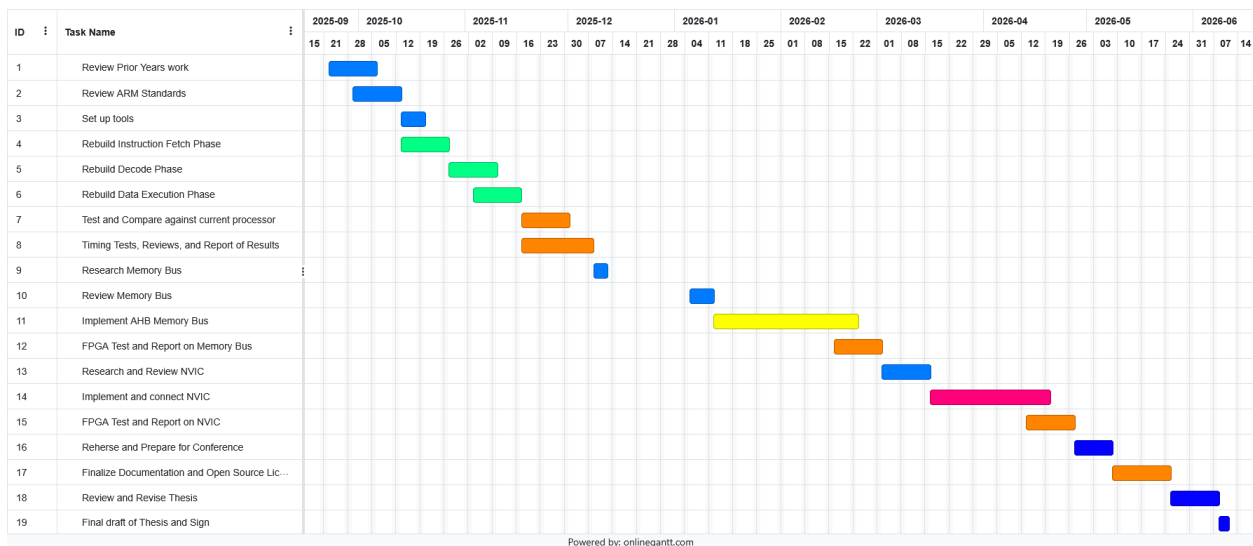


Figure 6.1: Fall 2025 quarter Gantt Chart of Timeline

Above is our planned timeline for our project from the Fall 2025 quarter. To start, we recovered as much of the code as we could from last year's SCU Senior Design Thesis on ARM Cortex-M0 by Zach Boyle [18]. Alongside that, we were researching the ARM Cortex-M datapath and Instruction Set to have baseline knowledge about the system before we began working. After discovering the prior code to be unrecoverable, we began work to rebuild the Core of the processor, doing each phase at a time and planning to end with a complete core which ran the basic register, memory, and branch instructions. In the following quarters, we followed a similar pattern while working on other functions of the processor. We started with the memory bus because it had more accessible information available before moving into the NVIC, which we believed would prove more difficult. In addition, we scheduled time between

working on different systems to ensure proper documentation and testing of each function. This ensured that each system had the required documentation, along with specifications about its function based on the tests we conducted. This also acted as quality assurance for each stage.

Overall, we stayed fairly close to our original timeline, with the main changes being a greater overlap between time spent developing FPGA tests and the individual components. This was done to increase the amount of testing and verification that we could do on our implementation.

6.2 Project Risks

Risk	Explanation	Mitigation
Missing Detection of Structural & Data Hazards	Structural and data hazards were a natural part of pipelined processors and had to be properly managed to ensure that all instructions ran in the correct order with the correct data. The risk was that since there were many instructions there may have been a certain hazard that went unnoticed which would cause the final processor to function differently compared to what was expected. The other risk was that the M0 might have had unique hazard resolutions compared to other processors which may have been difficult to replicate perfectly.	In order to mitigate this risk, we developed test sets that ran through not only every instruction, but multiple instructions with conflicting data and jump locations. We did this using available compilers to compile C programs into assembly, allowing for more complex tests.
Not Meeting AHB Timing Validation	Since the AHB-Lite was an already existing protocol we needed to recreate the bus matrix exactly, and due to limited documentation, it was difficult to find proper validation methods to ensure timing matched. Matching timing was important since this determined how the processor connected with peripherals, including memory, so slowdowns or improper timing here would drastically impact the processor's overall runtime.	To mitigate this risk, we first referenced the documentation of AHB-Lite to match the timing diagrams per clock cycle. This ensured our implementation was matching on a cycle-by-cycle basis. We then needed to test further by simulating the speed of our AHB implementation against the real speed of the current M0 AHB bus to ensure there was not a discrepancy.
Improper Interrupt Priority Logic or Fetching	The Cortex-M0's exact NVIC was not fully open for us to reference, so there was a risk that when developing our implementation we would have some level of logic that differed from the M0's NVIC. This risk mostly came from the lack of information that may have existed about the structure of the NVIC, making it more difficult to even know what the set priorities or logic should have been.	Our mitigation of this risk was to reference other current implementations of an NVIC unit and compare them to the physical M0's NVIC output. By running multiple tests with the real M0 we were able to develop an idea about the priority logic and by researching other existing implementations we were able to find and implement a fetching system with the same cycle times as expected by the M0.

Table 6.1: Risk Table

Risk	Explanation	Mitigation
Design isn't Synthesizable	In Verilog there was a difference between just a simulatable and a synthesizable design, where synthesis was more restrictive than just simulating. This presented the risk that since we were largely using simulation for testing, we may not have constructed a synthesizable design and not known.	Our main mitigation for this was to ensure we avoided using unsynthesizable structures in Verilog. Since there were set rules for what was and was not synthesizable, then as long as we followed those restrictions we should have been able to mitigate this risk. Occasional synthesis tests were also used to make sure we didn't go too far into implementation with an unsynthesizable design.
Not meeting Legal and Open-Source requirements	This risk came from adhering to IP, trademark, and other legal requirements which keeps our project fair use. With that, we needed to avoid copyright or patented information in both our documentation and implementation, as that private information was not allowed to be a part of an open-source distribution.	The mitigation for this was to ensure all of the documents or research we were using to make or base our design off of were open to the public. In this we wouldn't violate any ARM licensing while keeping where we got our development ideas from open to the public as documentation. Other than that, we never claim that our project is sponsored, endorsed, or officially validated through ARM and that it is instead a separate project. Finally we ensured all of the code we used was made originally by us, we did not use any already documented source code keeping the final design original.

Table 6.2: Risk Table Cont.

Chapter 7

Constraints and Standards

7.1 Constraints

7.1.1 Time

The largest constraint to our project was time. Due to the limited time of our project, we had to limit how many features of the Cortex-M0 we would attempt to implement. Time was especially limited, as for each potential component we would need to consider time for research, implementation, and testing or validation. With this, we were restricted to the minimal processor, which meant not implementing any of the other potential components such as WIC, JTAG, GPIO, or a Breakpoint unit. The minimal components were the most feasible due to more accessible resources defining them, and being what is actually needed for the processor to be a processor. With more time, we would be able to research and implement other components to create a more complete version of the processor.

7.1.2 Reliability

Reliability became a constraint since our system must work in the same way every time and have consistency in both function and timing. As a result, we needed to consider testing to be a large part of our available time along with which components we would create test suites for. Due to reliability constraints, we needed to ensure that any component we implemented was also tested and testable for various situations. We would then need to ensure that all of our components worked in both simulation and synthesized versions and were consistent with expected results on both platforms. [\[22\]](#)

7.1.3 Legal

Legal or IP requirements were a constant constraint with our project. Due to the nature of the project being an open-source version of an already existing product, it limited the scope of accessible resources along with how we would be able to use our product. This placed a restriction on how we could define our product within documentation as certain

terms or definitions are trademarked by ARM. Along with that, we could not use any non-public ARM documents or designs when writing our implementation. This restricted us to the publicly available ARMv6-M Reference Manual (Section 7.2.4) which had limited information on certain aspects of the processor. [15]

7.1.4 Available Documentation

Since our implementation is based on a closed-source product, it means many details about certain functions are very limited. This became a significant constraint when working on components outside of the processor core since many of these components didn't have much documentation describing their features or how they operated. This added to our time constraint as it meant research and validation of our system became more difficult due to the lack of available documents. In terms of validation, it restricted which tests we would be able to perform in order to ensure our system functioned properly due to documentation not defining its functions. Overall, the lack of available information became a constraint as we would be unable to find whether our implementation properly followed the design we wanted. [14]

7.2 Standards

7.2.1 Verilog

Verilog, IEEE 1364-2005, is an HDL (Hardware Description Language) which is used in the creation of electronic systems. It supports the development, synthesis, and testing of hardware designs in a human-readable format with various tools and compilers supporting the language. We used Verilog in our project to implement the entire codebase used for the processor. We also used compilation tools within the Vivado software to compile and synthesize our code. [20]

7.2.2 Synthesizable Verilog

While Verilog acts as an HDL, there is a difference in Verilog used for simulation and synthesis. Synthesizable Verilog, IEEE 1364.1-2005 has additional syntax and semantic requirements. As a result, Verilog RTL synthesis uses a subset of standard Verilog which is compatible with synthesis tools and programs. Since our product is meant to be implementable on an FPGA, it requires the Verilog to follow Verilog RTL synthesis standards so it may be synthesized. This is important for our open-source project as this standard ensures our implementation will be synthesizable across various software, making our product more portable. [21]

7.2.3 Cortex-M0 Technical Reference Manual

The definition of the Cortex-M0 main features and functions is within the Cortex-M0 Technical Reference Manual. It describes which instructions are part of the processor's ISA, their cycle counts, and which components are part of the processor, both optional and minimally required. Along with that, it details the potential configurable settings for the processor. This standard was used in our project to outline what was required for the scope of our product along with giving additional details on how our implementation should function. [14]

7.2.4 ARMv6-M

The ARMv6-M architecture defines the instruction set and overall CPU behavior. This includes all of the instructions' encoded values and their functions within the processor, memory descriptions, and component interactions within the system. We must adhere to this standard to ensure our Verilog model matches real Cortex-M0 functionality. As such, we used this standard throughout our project to both design and test our system to ensure that it followed the instruction definitions and expected results. [15]

7.2.5 AMBA 3 APB

AMBA 3 AHB-Lite defines the memory interface and communication protocols. This standard describes how the manager and subordinate modules of the memory/peripheral interface are expected to communicate and their structure. We used this standard in our project to design our memory implementation as this standard gives clear timing requirements for each memory access and peripheral call. [\[13\]](#)

Chapter 8

Societal Issues

8.1 Ethical

Publishing any CPU core can enable misuse and also expose design and security weaknesses that affect systems built on similar architectures and ISAs, and the M0 is no different for similar ARMv6 processors. Our project aims to expose the internals of the M0, and doing so may expose problems with the original design. However, we will not promote taking advantage of these risks in the original design. On top of that, exposure of these risks is important for those using the original design so that they are aware of any potential flaws and are able to mitigate them in their systems. Since we do not introduce new flaws into the design, exposing them may be important to bring awareness to the already present danger. Therefore, it is ethically important for us to call out these flaws so people can mitigate them. However, we will still need to make sure we are not promoting attacks against these potential risks.

Additionally, any mistakes we make can cause unpredictable behavior in downstream designs for those who implement our M0. To circumvent this, we have documented verification scope and discourage safety-critical use without extensive independent validation.

Another portion of security is in the GitHub itself for distributing the open-source implementation. This applies to the individual files posted on GitHub, and the people who have access to edit and make updates to the GitHub repository. For files, the main ethical concern is that no private data gets leaked through any means, making sure that all the documentation does not expose personal information. For access to the GitHub, we will need to maintain who has administrator or manager access, since this gives higher file-viewing privileges and permissions. These will need to be trusted individuals to ensure that people's information, which may be on GitHub, is not exposed and to make sure that our implementation does not get corrupted or manipulated to create new security threats.

8.2 Health and Safety

The health & safety risks associated with our project are practically zero. We focus purely on digital design, simulation, and synthesis. There are no hazardous systems, equipment, or chemicals. Any equipment we do use (computers, cables, FPGAs, etc.) will have to already have passed their own health & safety tests. The only potential issue arises from misuse or malfunction of our project in applications that involve health & safety. Since we are designing a processor that can be placed inside of larger systems, this means our processor must be properly detailed with accurate limitations or constraints. Within these constraints, our project must be reliable so that if it is used in a greater system, it won't fail and lead to potential safety concerns. Through proper reliability and documentation, we can avoid the misuse of our product and enhance safety through reliability, ensuring our system will not fail in safety-critical environments.

8.3 Manufacturability

Manufacturability was an important consideration during our project. Our project, as is, is merely synthesizable for FPGAs. We decided this would meet manufacturability standards since FPGAs are commonly used in real-world scenarios while being cost- and time-efficient during our development and testing of our design. The main restriction during development was ensuring our Verilog implementation would synthesize through common libraries into a reasonable design that met both area and timing constraints. The only cost during this process was the development time spent improving our simulation design and the single one-time cost of an FPGA board for testing.

8.4 Sustainability

In a narrow sense, our project is incredibly sustainable because it is designed to be useful and built upon after we complete it. All open-source files will be stored publicly, so the work does not just end there. Others can take our work and use it as is, or create something new. We are also documenting the project so adding or modifying the system does not require rewriting the whole project. This means the project will have some long-term use as a foundation for other projects or research into specifically the M0 or processors in general. Even as technology becomes more advanced, the files will be open to community growth, so the open-source model can be modified and expanded upon as desire or necessity changes.

In a broader sense, having the project open-source is sustainable as it reduces duplicated effort. Other students, professors, or engineers can reuse our modules, which saves time and resources when compared to remaking everything from scratch. This ties to the project being mainly software, which makes it fairly reusable with a low cost. This

contrasts with a typical physical chip which can degrade over time or need to be completely replaced should something happen to it or the project changes.

The only real sustainability risk we encounter is if our documentation does not describe our design in enough depth. This could limit others' understanding of our implementation, which could then lead to them having to allocate resources and time to try and understand, or alternatively start from scratch.

8.5 Environmental Impact

Our project is digital, so resources used to complete it are minimal. That said, there is still energy consumption related to running simulations and storing our files, albeit a small one.

If the design is deployed on hardware, there is a silicon usage aspect there as well. However, this is still far more efficient than taping out any sort of IC, and FPGAs are already going to be made. FPGAs are more efficient since they can have the design mapped or routed onto them multiple times, allowing for iterations and designs to be tested on the same piece of hardware rather than taping out, which is a set singular design. This means FPGAs save on hardware materials by being reusable while taping out is single-use, in terms of design or iteration. Since our project likely will not actively contribute to FPGA production, there is minimal environmental impact caused by manufacturing for our project.

Overall, our project uses energy and silicon to work, but is still implemented in one of the most environmentally friendly ways. Simply running the simulation only requires a pre-existing computer, software, and internet connection. Synthesizing on hardware requires an additional FPGA, which is already commercially produced and reusable for other logic circuits and projects.

8.6 Usability

The goal of our project is to be easily understandable and usable, requiring one click once the software is extracted from a public GitHub. Our implementation should be the bare Verilog necessary to simulate and synthesize the design, with public constraint files so others can synthesize easily without designing their own constraints. This makes our implementation easy to use as it will only require a simulation or synthesis program capable of running Verilog. Along with that, our Verilog has corresponding documentation to detail our specific design including pipelining details, timing and area reports, and instruction information. This documentation is segmented into specific sections and models to improve readability and help make our product more easily learned.

Chapter 9

Conclusion

Through this project, we have successfully researched and developed a processor core, AHB interface, and NVIC which are capable of behaviorally matching the ARM Cortex-M0's functions and timing. This implementation has been uploaded to the following GitHub repository <https://github.com/brkelly6585/Open-Source-ARM-Cortex-M0>. Using publicly available documents, our core is capable of running all ARMv6-M instructions with proper fetching and prefetching strategies along with cycle-accurate runtimes. The AHB interface is able to connect with peripherals following the same standard and accurately runs as a 2-stage pipeline. Finally, our NVIC is able to properly detect and send exceptions to the core, which can handle and return from them. We were able to connect these components together, creating a combined top module capable of running code just like the Cortex-M0 in both simulation and synthesized versions. We ran our system through various simulation and synthesized tests, testing each individual instruction in the ARMv6-M ISA and using a standard benchmark through CoreMark to verify and validate that our design is behaviorally accurate to the ARM Cortex-M0. Although behaviorally accurate, we have not confirmed whether our implementation is similar in terms of area or utilization. Along with that, our system only runs at a clock speed up to 40 MHz because faster clock speeds introduce timing violations, while the real Cortex-M0 can run up to 48 MHz. As a result, our project accurately represents the functionality and cycle times of the Cortex-M0 for research or education purposes, but it is not confirmed to reflect the exact processor's timing or area.

This opens our project to future work in properly replicating the timing and area of the Cortex-M0. Along with that, since our implementation focuses on the minimal functions, it leaves a lot to build upon. Future projects could finish the complete M0 with all optional components, such as debug lines and JTAG, or they could continue down the Cortex-M processors looking to implement more of the functions from the Cortex-M0+ or Cortex-M3 using our implementation as a base. Furthermore, research using this implementation can be done to look for improvements in the Cortex-M0's design, separate from those already made in the Cortex-M series. Overall, this project serves as a baseline minimal Cortex-M0 implementation such that there are many different options for future work to explore.

Chapter 10

Glossary of Abbreviations

This glossary defines abbreviations and project-specific terms used throughout the report.

AHB

Advanced High-performance Bus. In this project, AHB usually refers to the ARM AMBA bus protocol used by the Cortex-M0 memory and peripheral interface.

AHB-Lite

Advanced High-performance Bus Lite. A simplified single-manager version of AHB used by the Cortex-M0 for instruction, data, and peripheral accesses.

ALU

Arithmetic Logic Unit. The execution unit inside the processor core that performs arithmetic and logical operations such as addition, subtraction, comparisons, and bitwise operations.

AMBA

Advanced Microcontroller Bus Architecture. ARM's family of on-chip bus protocols, including AHB-Lite and APB.

APB

Advanced Peripheral Bus. A lower-complexity AMBA bus protocol commonly used for simple peripherals and control registers.

ARM

Advanced RISC Machines. In this report, ARM refers to the processor architecture family and related Cortex-M0 documentation used as the behavioral target for the implementation.

ARMv6-M

ARM version 6-M. The architecture profile implemented by the Cortex-M0, defining the instruction behavior, exception model, programmer's model, and memory behavior used by this project.

BLX

Branch with Link and Exchange. An ARM instruction mnemonic for branching while saving a return address and potentially changing instruction set state. In this Cortex-M0 project, it appears in instruction decode logic.

BX

Branch and Exchange. An ARM instruction mnemonic for branching to an address held in a register. In this Cortex-M0 project, it appears in instruction decode logic.

CMP

Compare. An ARM instruction mnemonic that subtracts values to update condition flags without writing the subtraction result to a register.

CoreMark

A processor benchmark maintained by EEMBC. It is used in this project to compare the synthesized Cortex-M0 implementation against expected embedded-processor behavior.

CPU

Central Processing Unit. The main processing unit in a computer system that executes instructions, performs arithmetic, and coordinates system activity.

CSEN

Computer Science and Engineering. Used for Santa Clara University department and course references.

ECEN

Electrical and Computer Engineering. Used for Santa Clara University department and course references.

EEMBC

Embedded Microprocessor Benchmark Consortium. The organization associated with the CoreMark embedded processor benchmark.

FPGA

Field-Programmable Gate Array. Reconfigurable hardware used to synthesize and physically test the Verilog Cortex-M0 implementation.

GPIO

General-Purpose Input/Output. Configurable digital input/output pins or peripheral interfaces used in many embedded systems.

GPLv3

GNU General Public License version 3. The open-source license used for the public repository described in this project.

HDL

Hardware Description Language. A language used to describe digital hardware. Verilog is the HDL used for this Cortex-M0 implementation.

HADDR

AHB address signal. The bus signal that carries the address for an AHB-Lite transfer.

HCLK

AHB clock signal. The clock used to synchronize AHB-Lite bus transfers and many project modules.

HRDATA

AHB read-data signal. The bus signal carrying data returned from a subordinate to the manager during a read transfer.

HREADY

AHB ready signal. Indicates that the current AHB-Lite transfer can complete and that the bus can advance.

HREADYOUT

AHB subordinate ready-output signal. Indicates whether a selected subordinate is ready to complete a transfer.

HRESETn

Active-low AHB reset signal. The reset signal used in several Verilog modules and testbenches.

HRESP

AHB response signal. Indicates whether an AHB-Lite transfer completed successfully or returned an error response.

HSEL

AHB subordinate select signal. Indicates that a particular subordinate device or memory region has been selected for a transfer.

HSIZE

AHB transfer-size signal. Encodes the size of the current transfer, such as byte, halfword, or word.

HTRANS

AHB transfer-type signal. Identifies the type of AHB-Lite transfer, such as idle, busy, nonsequential, or sequential.

HWDATA

AHB write-data signal. The bus signal carrying data from the manager to the selected subordinate during a write transfer.

HWRITE

AHB write-control signal. Indicates whether the current AHB-Lite transfer is a read or a write.

IC

Integrated Circuit. A fabricated electronic circuit on silicon. In this project, IC appears mainly in comparison to FPGA implementation and possible future silicon implementation.

IDE

Integrated Development Environment. Software used for editing, compiling, simulating, debugging, or programming the design. Vivado and Keil are examples used in this project.

IFID

Instruction-Fetch/Instruction-Decode buffer. A pipeline register between the fetch and decode stages in the project implementation.

IPSR

Interrupt Program Status Register. An ARM special-purpose register that records the currently active exception number.

IRQ

Interrupt Request. A hardware or software event that requests interrupt handling by the processor.

ISA

Instruction Set Architecture. The programmer-visible instruction behavior and architectural rules that the processor must implement. This project targets the ARMv6-M ISA.

JTAG

Joint Test Action Group. A common hardware debug and test interface. It is mentioned as an optional or future debug-related feature.

LR

Link Register. ARM register R14, commonly used to store a return address for function calls or exception returns.

M0

Cortex-M0. The specific ARM Cortex-M processor used as the target for this open-source Verilog implementation.

MIPS

Microprocessor without Interlocked Pipeline Stages. A RISC processor architecture mentioned as a comparison point to ARM and RISC-V.

MOV

Move. An ARM instruction mnemonic used to copy or load a value into a register.

NOP

No Operation. An instruction that intentionally performs no state-changing operation, often used for pipeline control or alignment.

NVIC

Nested Vectored Interrupt Controller. The Cortex-M interrupt controller responsible for enabling, pending, prioritizing, and signaling exceptions and external interrupts.

PC

Program Counter. ARM register R15, which holds the address of the current or next instruction being executed.

PRIMASK

Priority Mask Register. An ARM special-purpose register that can mask configurable exceptions and interrupts.

REG

Register. In the appendix, REG appears in instruction-decode comments and refers to register-based instruction forms.

RISC

Reduced Instruction Set Computer. A processor design style based on a relatively small and regular instruction set.

RISC-V

Reduced Instruction Set Computer Five. An open instruction set architecture mentioned as a comparison point because of its open-source ecosystem.

RTL

Register-Transfer Level. A hardware design abstraction describing data movement between registers and combinational logic. The project implementation is written in synthesizable Verilog RTL.

SCB

System Control Block. A Cortex-M system-control register block used for exception status, system reset, sleep behavior, and other system-level controls.

SCS

System Control Space. The Cortex-M memory region used for processor-internal system registers such as NVIC, SCB, and SysTick registers.

SEVONPEND

Send Event on Pending bit. A Cortex-M system-control setting that can cause an event to be generated when an interrupt becomes pending.

SLEEPDEEP

Sleep Deep bit. A Cortex-M system-control setting used to request deeper low-power sleep behavior.

SLEEPONEXIT

Sleep on Exit bit. A Cortex-M system-control setting that causes the processor to enter sleep after returning from an exception handler.

SRAM

Static Random-Access Memory. A memory type commonly used for embedded data memory.

STM

STMicroelectronics. The project uses an STM32F0 Discovery Board for comparison against a commercial Cortex-M0 implementation.

STM32F0

STMicroelectronics 32-bit microcontroller family based on the ARM Cortex-M0. This board family is used for real-hardware comparison testing.

SUB

Subtract. An ARM instruction mnemonic and ALU operation used for subtraction.

SysTick

System Tick Timer. A Cortex-M timer intended for periodic interrupts, often used by embedded operating systems or timing routines.

SYSRESETREQ

System Reset Request bit. A Cortex-M system-control signal or register bit used to request a system reset.

TRM

Technical Reference Manual. ARM documentation that describes Cortex-M0 processor behavior, components, configuration options, and implementation details.

UML

Unified Modeling Language. A diagramming notation used for design documentation, including use-case, class, activity, and sequence diagrams.

VECTCLRACTIVE

Vector Clear Active bit. A Cortex-M debug or system-control related bit that can clear active exception state in certain contexts.

WIC

Wake-up Interrupt Controller. An optional Cortex-M component that can detect interrupts while the processor is in low-power sleep states.

Bibliography

- [1] Coremark benchmark source-code github, . URL <https://github.com/eembc/coremark>.
- [2] Coremark benchmark informational website, . URL <https://www.eembc.org/coremark/>.
- [3] Keil embedded development tools. URL <https://www.keil.com/>.
- [4] Microcontroller (mcu) market size & share analysis - growth trends & forecasts. URL <https://www.mordorintelligence.com/industry-reports/microcontroller-mcu-market>.
- [5] Arm launches its smallest, lowest power, most energy efficient processor, 9 2009. URL <https://www.design-reuse.com/news/202516316-arm-launches-its-smallest-lowest-power-most-energy-efficient-processor/>.
- [6] Vectored interrupt controller (nested vectored interrupt controller) vic/nvic, 2020. URL <https://embetronicx.com/tutorials/microcontrollers/stm32/vectored-interrupt-controller-nested-vectored-interrupt-controller-vic-nvic/>.
- [7] Applications of embedded systems, 2021. URL <https://www.rs-online.com/designspark/applications-of-embedded-systems-1>.
- [8] Arm silicon ip: Cpu processors, 2026. URL <https://www.arm.com/products/silicon-ip-cpu>.
- [9] Arm cortex-m0 processor, 2026. URL <https://www.arm.com/products/silicon-ip-cpu/cortex-m/cortex-m0>.
- [10] Nexys a7: Fpga trainer board, 2026. URL <https://digilent.com/reference/programmable-logic/nexys-a7/start>.
- [11] Stm32f0discovery evaluation board overview, 2026. URL <https://www.st.com/en/evaluation-tools/stm32f0discovery.html#overview>.

- [12] Vivado design suite, 2026. URL <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html>.
- [13] *AMBA 3 APB Protocol Specification*. ARM Ltd., 2004. URL <https://developer.arm.com/documentation/ihl0024/latest>. Document Number: ARM IHI 0024B.
- [14] *ARM Cortex-M0 Technical Reference Manual*. ARM Ltd., 2009. URL <https://developer.arm.com/documentation/ddi0432/latest>. Document Number: ARM DDI 0432C.
- [15] *ARMv6-M Architecture Reference Manual*. ARM Ltd., issue e.b edition, 2010. URL <https://developer.arm.com/documentation/ddi0419/latest>. Document Number: ARM DDI 0419E.
- [16] *ARM Cortex-M0 DesignStart RTL Testbench User Guide*. Arm Ltd., 2022. URL <https://developer.arm.com/documentation/dui0926/a>.
- [17] *ARM Cortex-M0 Processor Datasheet*. Arm Ltd., 2026. URL <https://developer.arm.com/documentation/>. Processor Datasheet.
- [18] Zack Boyle. Open source arm cortex m0 processor, 2025. URL https://scholarcommons.scu.edu/elec_senior/109.
- [19] John L. Hennessy and David A. Patterson. Appendix k: Survey of risc architectures for desktop, server, and embedded computers. URL https://zoo.cs.yale.edu/classes/cs323/CAAQA6E/Appendix_K_online.pdf.
- [20] *IEEE Standard for Verilog Hardware Description Language*. IEEE, 2005. URL <https://standards.ieee.org/ieee/1364/3641/>.
- [21] *IEC/IEEE International Standard - Verilog(R) Register Transfer Level Synthesis*. IEEE, 2005. URL <https://standards.ieee.org/ieee/62142/4137/>.
- [22] *ISO/IEC/IEEE 29148: Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*. ISO/IEC/IEEE, 2018. URL <https://www.iso.org/standard/72089.html>. Defines reliability as a required quality attribute in system requirements.
- [23] *Walking Ones Test*. NXP Semiconductors, 2015. URL <https://docs.nxp.com/bundle/GUID-9FAC1C79-3809-474F-B8DF-82BEB5B88419/page/GUID-90DC350A-C37F-44C9-A785-1E09D86D5BAA.html>. Code-Warrior Development Studio for Power Architecture Technology Documentation.

- [24] Staff Writer. What are embedded systems and how do they work?, 3 2019. URL <https://www.totalphase.com/blog/2019/03/what-are-embedded-systems/>.
- [25] Joseph Yiu. *The Definitive Guide to ARM Cortex-M0 and Cortex-M0+ Processors*. Newnes, Oxford, U.K., 2 edition, 2015. Elsevier. ISBN: 978-0-12-803277-0. DOI: 10.1016/B978-0-12-803277-0.01001-8.

Chapter 11

Appendices

11.1 Source Code

11.1.1 M0 Top Module

```
'timescale 1ns / 1ps

module M0_top(
);

    reg HCLK, HRESETn;

    wire      ahb_ready, ahb_error;
    wire [31:0] ahb_rdata;

    wire      ahb_read, ahb_write;
    wire [1:0] ahb_size;
    wire [31:0] ahb_addr;
    wire [31:0] ahb_wdata;

    wire [31:0] cpu_addr;
    wire [31:0] cpu_wdata;
    wire      cpu_wr, cpu_rd;
    wire [31:0] nvic_rdata, scb_rdata, syst_rdata;

    wire ppb_sel      = (cpu_addr[31:12] == 20'hE000E);
    wire systick_sel   = ppb_sel && (cpu_addr[11:8] == 4'h0) && (cpu_addr[7:4] != 4'h0); // 0x010-0x0FF
    wire nvic_sel      = ppb_sel && (cpu_addr[11:8] >= 4'h1) && (cpu_addr[11:8] < 4'hD);
    wire scb_sel       = ppb_sel && (cpu_addr[11:8] == 4'hD);

    wire [31:0] ppb_rdata = nvic_sel      ? nvic_rdata :
                           scb_sel       ? scb_rdata  :
                           systick_sel   ? syst_rdata :
                           32'b0;

    wire      nmi_pend, hardfault_pend;
    wire      svcall_pend, pendsv_pend, systick_pend;
    wire [1:0] svcall_pri, pendsv_pri, systick_pri;
```

```

wire      vectclractive;
wire      isrpending;
wire [8:0] vectpending;

wire      exc_taken,      exc_return;
wire [5:0] exc_taken_num, exc_return_num;
wire      svcall_req;
wire      fault_req;
wire [5:0] IPSR;
wire      PRIMASK;
wire      core_halted;
wire      systick_fire;

wire      int_pend;
wire [5:0] int_pend_num;

wire nmi_i;
wire [31:0] gpio_irq;
wire SLEEPONEXIT, SLEEPDEEP, SEVONPEND, SYSRESETREQ;

initial begin
    HCLK = 0;
    forever #5 HCLK = ~HCLK;
end

core core_dut(
    .HCLK(HCLK),
    .HRESETn(HRESETn),

    .rdata_i(ahb_rdata),
    .ready_i(ahb_ready),
    .error_i(ahb_error),

    .addr_o(ahb_addr),
    .wdata_o(ahb_wdata),
    .read_req_o(ahb_read),
    .write_req_o(ahb_write),
    .size_o(ahb_size),

    .ppb_addr_o      (cpu_addr),
    .ppb_wdata_o      (cpu_wdata),
    .ppb_wr_o         (cpu_wr),
    .ppb_rd_o         (cpu_rd),
    .ppb_rdata_i      (ppb_rdata),

    .exc_taken_o      (exc_taken),
    .exc_taken_num_o(exc_taken_num),
    .exc_return_o      (exc_return),
    .exc_return_num_o(exc_return_num),
    .svcall_req_o      (svcall_req),
    .fault_req_o       (fault_req),

```

```

        .IPSR_o      (IPSR),
        .PRIMASK_o   (PRIMASK),
        .core_halted_o (core_halted),

        .int_pend_i   (int_pend),
        .int_pend_num_i (int_pend_num),

        .SLEEPONEXIT_i (SLEEPONEXIT),
        .SLEEPDEEP_i   (SLEEPDEEP),
        .SEVONPEND_i   (SEVONPEND)
    );

    ahb_top ahb(
        .HCLK(HCLK),
        .HRESETn(HRESETn),

        .addr_i(ahb_addr),
        .wdata_i(ahb_wdata),
        .read_req_i(ahb_read),
        .write_req_i(ahb_write),
        .size_i(ahb_size),

        .rdata_o(ahb_rdata),
        .ready_o(ahb_ready),
        .error_o(ahb_error)
    );

    NVIC NVIC_Unit(
        .clk      (HCLK),
        .rst_n    (HRESETn),

        .reg_addr (cpu_addr[11:0]),
        .reg_wdata (cpu_wdata),
        .reg_wr    (cpu_wr & nvic_sel),
        .nvic_rdata (nvic_rdata),

        .irq_i    (gpio_irq),

        .PRIMASK   (PRIMASK),
        .exc_taken (exc_taken),
        .exc_taken_num (exc_taken_num),
        .exc_return (exc_return),
        .exc_return_num (exc_return_num),

        .nmi_pend  (nmi_pend),
        .hardfault_pend (hardfault_pend),
        .svcall_pend (svcall_pend),
        .pendsv_pend (pendsv_pend),
        .systick_pend (systick_pend),
        .svcall_pri (svcall_pri),
        .pendsv_pri (pendsv_pri),

```

```

        .systick_pri      (systick_pri),
        .VECTCLRACTIVE   (vectclractive),

        .int_pend         (int_pend),
        .int_pend_num     (int_pend_num),

        .isrpending       (isrpending),
        .vectpending      (vectpending)
    );

    SysTick SysTick_Unit (
        .clk               (HCLK),
        .rst_n             (HRESETn),

        .reg_addr          (cpu_addr[11:0]),
        .reg_wdata         (cpu_wdata),
        .reg_wr            (cpu_wr & systick_sel),
        .reg_rd            (cpu_rd & systick_sel),
        .syst_rdata        (syst_rdata),

        .core_halted       (core_halted),

        .systick_fire      (systick_fire)
    );

    SCB SCB_Unit (
        .clk               (HCLK),
        .rst_n             (HRESETn),

        .reg_addr          (cpu_addr[11:0]),
        .reg_wdata         (cpu_wdata),
        .reg_wr            (cpu_wr & scb_sel),
        .scb_rdata         (scb_rdata),

        .nmi_i             (nmi_i),

        .IPSR              (IPSR),
        .exc_taken         (exc_taken),
        .exc_taken_num     (exc_taken_num),
        .svcall_req        (svcall_req),
        .fault_req         (fault_req),
        .systick_fire      (systick_fire),

        .isrpending        (isrpending),
        .vectpending       (vectpending),
        .nmi_pend          (nmi_pend),
        .hardfault_pend    (hardfault_pend),
        .svcall_pend       (svcall_pend),
        .pendsv_pend       (pendsv_pend),
        .systick_pend      (systick_pend),
        .svcall_pri        (svcall_pri),

```

```

        .pendsv_pri      (pendsv_pri),
        .systick_pri    (systick_pri),
        .VECTCLRACTIVE  (vectclractive),

        .SYSRESETREQ    (SYSRESETREQ),
        .SLEEPONEXIT    (SLEEPONEXIT),
        .SLEEPDEEP      (SLEEPDEEP),
        .SEVONPEND      (SEVONPEND)
    );

    initial begin
        HRESETn = 1'b0;
        #10
        HRESETn = 1'b1;
        #2000
        $finish;
    end

endmodule

```

11.1.2 IFID Buffer - Example Buffer

```

module IFID(
    input clk,
    input IFID_EN,
    input IFID_consume,
    input IFID_wipe,
    input [31:0] PC_in, Instr_in,
    output reg consumed,
    output reg [31:0] PC_out, Instr_out
);
    parameter [15:0] NOP = 16'hBF00;
    always @(negedge clk) begin
        if (IFID_wipe) begin
            PC_out    <= 32'hFFFF;
            Instr_out <= {NOP, NOP};
            consumed  <= 1'b1;
        end else if (IFID_EN && (PC_in != PC_out)) begin
            PC_out    <= PC_in;
            Instr_out <= Instr_in;
            consumed  <= 1'b0;
        end else if (IFID_consume && !consumed) begin
            // Used to track when prefetch is fully consumed
            if (PC_out[1]) begin
                consumed <= 1'b1;
                Instr_out <= {NOP, NOP};
            end else if (IFID_EN) begin
                // If the buffer is enabled, but PC hasn't moved due to fetch,
                // Can still inc by 2 to process next half
                PC_out    <= PC_out + 32'd2;
                consumed <= 1'b0;
            end
        end
    end
endmodule

```

```

        end
    end else if (consumed) begin
        // Wait for new fetch
        Instr_out <= {NOP, NOP};
    end
end
endmodule

```

11.1.3 Special Data Instruction Decoding - Example Decode

```

6'b010001: begin //Special Data Instruct
    Rm = Instr16b[6:3];
    Rd = {Instr16b[7],Instr16b[2:0]};
    Rn = {Instr16b[7],Instr16b[2:0]};
    ALU_src = 1'b0;
    memread = 1'b0;
    memwrite = 1'b0;
    regwrite = 1'b1;
    wd_src = 1'b0;
    move = 1'b0;
    flags = 1'b1;

    casex(Instr16b[9:6])
        4'b00xx: begin // ADD REG
            ALU_op = ADD;
        end
        4'b0100: begin // UNPREDICTABLE

    end
        4'b0101: begin // CMP REG (T2) R0-R7
            regwrite = 1'b0;
            ALU_op = SUB;
        end
        4'b011x: begin // CMP REG (T2) not R0-R7 basically don't allow PC
            regwrite = 1'b0;
            ALU_op = SUB;
        end
        4'b10xx: begin // MOV REG
            ALU_op = ADD;
            ALU_src = 1'b0;
            Imm = 32'd0;
            move = 1'b1;
            flags = 1'b0;
        end
        4'b110x: begin // BX
            regwrite = 1'b0;
            flags = 1'b0;
            branchC = 1'b1;
            cond = 4'b1111;
        end
        4'b111x: begin // BLX  //// SET Rd = LR; Rn = PC; Imm = 1

```



```

        ALU_op = SUB;
        Rn = 4'hF;
        Rd = 4'hE;
        Imm = 1;
        regwrite = 1'b1;
        flags = 1'b0;
        branchC = 1'b1;
        cond = 4'b1111;
    end
endcase

```

```

end

```

11.1.4 AHB Subordinate/Secondary Module

```

module ahb_second (
    input wire HCLK,
    input wire HRESETn,

    // AHB Slave Inputs
    input wire HSEL,
    input wire [31:0] HADDR,
    input wire [1:0] HTRANS,
    input wire HWRITE,
    input wire [2:0] HSIZE,
    input wire [31:0] HWDATA,
    input wire HREADY,

    // AHB Slave Outputs
    output wire [31:0] HRDATA,
    output wire HREADYOUT,
    output wire HRESP,

    // Memory Interface
    output wire mem_write, // 1 = write
    output wire [31:0] mem_raddr,
    output wire [31:0] mem_waddr,
    output wire [31:0] mem_wdata,
    output wire [3:0] mem_byte_en,
    input wire [31:0] mem_rdata
);

// LOCAL PARAMETERS
localparam [1:0] HTRANS_IDLE = 2'b00;
localparam [1:0] HTRANS_NONSEQ = 2'b10;

localparam [2:0] HSIZE_BYTE = 3'b000;
localparam [2:0] HSIZE_HALFWORD = 3'b001;
localparam [2:0] HSIZE_WORD = 3'b010;

```

```

// INTERNALS
reg [31:0] HADDR_reg;
reg [2:0] HSIZE_reg;
reg HWRITE_reg;
reg [31:0] HWDATA_reg;
reg valid_reg;

// Valid transfer detection
wire valid_transfer = HSEL && HREADY && (HTRANS == HTRANS_NONSEQ);

// ADDRESS PHASE REGISTER
always @(posedge HCLK or negedge HRESETn) begin
    if (!HRESETn) begin
        HADDR_reg <= 32'h0;
        HWDATA_reg <= 32'h0;
        HSIZE_reg <= HSIZE_WORD;
        HWRITE_reg <= 1'b0;
        valid_reg <= 1'b0;
    end
    else if (HREADY) begin
        if (valid_transfer) begin
            HADDR_reg <= HADDR;
            HSIZE_reg <= HSIZE;
            HWRITE_reg <= HWRITE;
            HWDATA_reg <= HWDATA;
        end
        valid_reg <= valid_transfer;
    end
end

// Byte enabling for writes
reg [3:0] byte_en;

always @(*) begin
    byte_en = 4'b0000;
    if (valid_reg && HWRITE_reg) begin
        case (HSIZE_reg)
            HSIZE_BYTE: begin
                case (HADDR_reg[1:0])
                    2'b00: byte_en = 4'b0001;
                    2'b01: byte_en = 4'b0010;
                    2'b10: byte_en = 4'b0100;
                    2'b11: byte_en = 4'b1000;
                endcase
            end
            HSIZE_HALFWORD: begin
                case (HADDR_reg[1])
                    1'b0: byte_en = 4'b0011;
                    1'b1: byte_en = 4'b1100;
                endcase
            end
        endcase
    end
end

```

```

        HSIZE_WORD: byte_en = 4'b1111;
        default: byte_en = 4'b1111;
    endcase
end
end

// MEMORY OUTPUTS
assign mem_raddr = HADDR; // added
assign mem_waddr = HADDR_reg; // was mem_addr
assign mem_wdata = valid_reg ? HWDATA : HWDATA_reg;
assign mem_byte_en = byte_en;
assign mem_write = valid_reg && HWRITE_reg;

// AHB OUTPUTS
assign HRDATA = mem_rdata;
assign HREADYOUT = 1'b1; // always one because mem is zero wait state
assign HRESP = 1'b0; // These slaves dont give errors
endmodule

```

11.2 Top Level RTL Synthesis

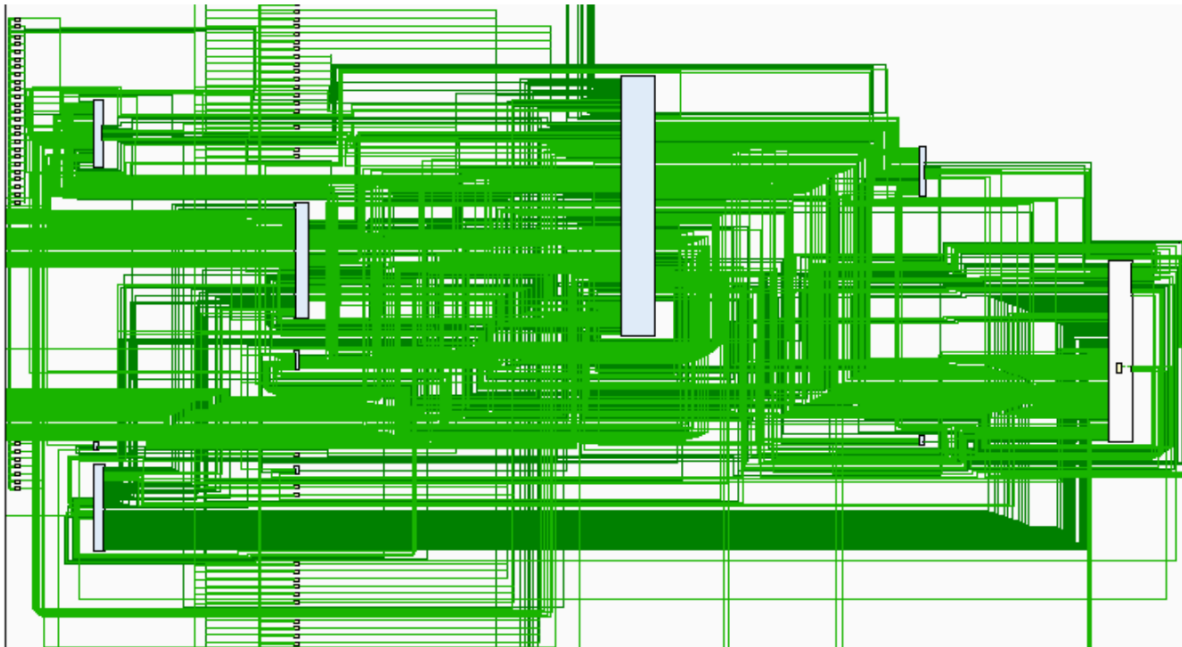


Figure 11.1: Top Level RTL synthesis from Vivado